

A

Major Project Report

On

**ENHANCING INDUSTRIAL WIRELESS SENSOR
NETWORK SECURITY USING MACHINE LEARNING
AND DEEP LEARNING MODELS**

Submitted to CMR Engineering College

In

Partial Fulfillment of the requirements for the Award of Degree of
BACHELOR OF TECHNOLOGY

IN

CSE-CYBER SECURITY

Submitted

By

B. Kartheek (228R1A6213)

A. Shivanand (228R1A6207)

MD. Amer (228R1A6247)

Under the Esteemed guidance of

Dr.X.S.Asha Shiny

Professor, Department of IT



Department of CSE-CYBER SECURITY
CMR ENGINEERING COLLEGE
(UGC AUTONOMOUS)

(Approved by AICTE, NEW DELHI, Affiliated to JNTU, Hyderabad)
Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)

2025-2026

CMR ENGINEERING COLLEGE

(UGC AUTONOMOUS)

(Accredited by NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU, Hyderabad)

Kandlakoya, Medchal Road, Hyderabad-501 401

Department of CSE-CYBER SECURITY



CERTIFICATE

This is to certify that the project entitled “**AI for Cybersecurity: From Adversarial Anomaly Detection to Intelligent Network Security systems (phishing URL)**” is a bonafide work carried out by

B. Kartheek (228R1A6213)

A. ShivaNand (228R1A6207)

MD. Amer (228R1A6247)

In partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **CSE-CYBER SECURITY** from CMR Engineering College, affiliated to JNTU, Hyderabad, under our guidance and supervision.

The results presented in this project have been verified and are found to be satisfactory. The results embodied in this project have not been submitted to any other university for the award of any other degree or diploma.

Internal Guide

Dr.X.S.Asha Shiny

Professor

Department of IT
CMREC

PRC

Dr.X.S.Asha Shiny

Professor

Department of IT
CMREC

Head of the Department

Dr. E. Suresh Babu

Associate Professor & HoD

CSE-CS Department
CMREC

External Examiner

DECLARATION

This is to certify that the work reported in the present project entitled “**AI for Cybersecurity: From Adversarial Anomaly Detection to Intelligent Network Security systems (phishing URL)**” is a record of bonafide work done by us in the Department of CSE- Cyber Security, CMR Engineering College, Hyderabad. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

B. Kartheek	(228R1A6213)
A. Shivanand	(228R1A6207)
MD. Amer	(228R1A6247)

ACKNOWLEDGMENT

We are extremely grateful to **Dr. A.Srinivasula Reddy**, Principal and **Dr. E. Suresh Babu**, **Associate Professor & HoD, Department of CSE-CS, CMR Engineering College** for their constant support.

We are extremely thankful to **Dr.X.S. Asha Shiny, Professor, Internal Guide**, Department of IT, for his constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if we do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We thank **Dr.X.S. Asha Shiny, Professor, PRC** for her constant support in carrying out the project activities and reviews.

We express our thanks to all staff members and friends for all the help and co- ordination extended in bringing out this project successfully in time.

Finally, We are very much thankful to my parents who guided us for every step.

B. Kartheek	(228R1A6213)
A. Shivanand	(228R1A6207)
MD. Amer	(228R1A6247)

CONTENTS

TOPIC	PAGE NO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
1. INTRODUCTION	1
1.1 Introductionof the project	1
1.2 Purpose of the Project	2
1.3 Objective	3
1.4 Existing System	3
1.4.1Disadvantages	4
1.5 Proposed System	5
1.5.1Advantages	5
2. LITERATURE SURVEY	7
3. SYSTEM ANALYSIS	16
3.1. Problem Specification	16
3.2. Modules and Their Functions	17
3.3. Functional Requirements	18
3.4. Non functional Requirements	19
3.5. Feasibility analysis	19
4. SYSTEM REQUIREMENTS	21
4.1. System Specification	21
4.2. Software Requirements	22
4.3. Hardware Requirements	23
5. SOFTWARE DESIGN	25
5.1. System Architecture	25
5.2. Data Flow Diagram	27

6. UML DIAGRAMS	29
6.1. Use case Diagram	29
6.2. Activity Diagram	32
6.3. Sequence Diagram	35
6.4. Class Diagram	
7. CODING AND IMPLEMENTATION	41
8. SYSTEM TESTING	53
8.1 Unit Testing	53
8.2 Functional Testing	55
8.3 Boundary Value Testing	56
8.4 Edge Case Testing	56
8.5 Black Box Testing	57
8.6 White Box Testing	58
8.7 Integration Testing	58
8.8 User Acceptance Testing	59
8.9 Performance Testing	60
9. OUTPUT SCREENS	61
10. CONCLUSION	69
11. FUTURE ENHANCEMENTS	71
12. REFERENCES	76

ABSTRACT

In the smart industrial era, modern industries rely heavily on industrial communication networks that connect machines, devices, and digital systems for real-time automation, monitoring, and decision-making. While this connectivity significantly boosts operational efficiency, it also exposes industrial environments to a wide range of sophisticated cybersecurity threats such as Insider Attacks, Advanced Persistent Threats (APT), Ransomware, and Denial of Service (DoS) attacks. These threats can disrupt operations, compromise sensitive information, and result in severe financial and reputational damage.

Existing intrusion detection systems in the base research primarily use AI, Machine Learning (ML), and Deep Learning (DL) techniques such as Decision Trees, Multi-Layer Perceptron (MLP), and Autoencoders. Although these models offer useful intrusion detection capabilities, they rely only on publicly available datasets, which limits their ability to identify real-world or previously unseen attack types. Furthermore, their limited scalability restricts performance when dealing with large, diverse, and continuously evolving industrial datasets, causing reduced effectiveness in practical deployment.

To overcome these challenges, this study proposes a hybrid intrusion detection model that integrates Autoencoders with XGBoost, enabling efficient analysis of real-world industrial data. The Autoencoder module improves data quality by extracting meaningful latent features, while XGBoost provides strong classification performance for detecting various types of cyberattacks. This hybrid approach enhances the detection of unseen and emerging threats, supports scalability for high-volume industrial data streams, and improves overall accuracy and system stability. Additionally, the model strengthens trust, transparency, and accountability in incident response through interpretable decision outputs and reliable alert mechanisms.

Overall, the proposed system aims to offer a more adaptive, scalable, and robust intrusion detection solution suitable for next-generation smart industrial environments, delivering stronger cybersecurity protection in an increasingly interconnected industrial landscape.

INDEX TERMS: Cybersecurity, detection, prediction, intrusions, machine learning and deep learning.

LIST OF FIGURES

SL.NO	FIG.NO	DESCRIPTION	PAGE.NO
1	5.1.1	System Architecture Diagram	25
2	5.2.1	Dataflow Diagram	27
3	6.1.1	Use Case Diagram	29
4	6.2.1	Activity Diagram	32
5	6.3.1	Sequence Diagram	35
6	6.4.1	Class Diagram	37
7	9.1	Main interface of the Network Intrusion Detection System	58
8	9.2	Uploading the NSL-KDD dataset	59
9	9.3	Dataset preprocessing and cleaned data representation	59
10	9.4	Training and testing model generation	60
11	9.5	Intrusion detection result using SVM algorithm	61
12	9.6	Intrusion detection result using ANN algorithm	61
13	9.7	Intrusion detection result using Logistic Regression algorithm	62
14	9.8	Intrusion detection result using Decision Tree algorithm	62
15	9.9	Intrusion detection result using Random Forest algorithm	63
16	9.10	Test data upload and intrusion detection results	63
17	9.11	Accuracy comparison of machine learning algorithms	64

1. INTRODUCTION

The rapid advancement of digital technologies has transformed traditional industries into highly automated and interconnected systems known as smart industrial environments. These environments rely on industrial communication networks that enable seamless interaction between machines, sensors, controllers, and enterprise systems. Technologies such as Industrial Internet of Things (IIoT), cloud computing, and artificial intelligence play a major role in improving productivity, operational efficiency, and real-time decision-making.

However, this increased connectivity has also expanded the attack surface for cyber threats. Industrial systems, which were once isolated, are now connected to external networks, making them vulnerable to various types of cyberattacks. These include insider threats, advanced persistent threats (APT), ransomware attacks, and denial-of-service (DoS) attacks. Such attacks can disrupt critical operations, compromise sensitive data, and cause financial and reputational damage.

To address these challenges, intrusion detection systems (IDS) have been developed to monitor network traffic and detect malicious activities. Traditional IDS approaches are not sufficient to handle modern, sophisticated attacks, leading to the adoption of AI, machine learning, and deep learning techniques. Despite these advancements, existing systems face limitations in scalability, adaptability, and detection of unseen threats.

This project focuses on developing a hybrid intrusion detection system that integrates Autoencoders and XGBoost to enhance detection accuracy, scalability, and real-world applicability in industrial environments.

1.1 Introduction of Project

The project titled “Hybrid Intrusion Detection System for Smart Industrial Networks using Autoencoder and XGBoost” aims to provide an advanced cybersecurity solution for modern industrial environments. With the increasing adoption of smart technologies, industrial systems are becoming more complex and interconnected, requiring robust security mechanisms to protect against evolving cyber threats.

This project introduces a hybrid approach that combines deep learning and machine learning techniques to improve intrusion detection performance. The Autoencoder is used for extracting meaningful features from raw industrial data by learning hidden patterns and reducing dimensionality. These extracted features are then used by the XGBoost algorithm, which performs efficient classification of network traffic into normal and malicious categories.

The system is designed to detect multiple types of cyberattacks, including known and unknown threats. It focuses on improving detection accuracy, reducing false positives, and enhancing the system's ability to handle large volumes of data. The project also emphasizes scalability and adaptability, making it suitable for real-time industrial applications.

By integrating advanced techniques and addressing the limitations of existing systems, this project contributes to the development of a more reliable and intelligent intrusion detection system.

1.2 Purpose of Project

The purpose of this project is to develop a robust and scalable intrusion detection system that can effectively protect smart industrial environments from cyber threats. As industries become more dependent on digital communication networks, the need for advanced security solutions has become critical.

One of the key purposes of this project is to overcome the limitations of traditional intrusion detection systems, which are unable to detect unknown or evolving attacks. By using a hybrid model that combines Autoencoders and XGBoost, the system aims to improve detection accuracy and adaptability.

Another important purpose is to enhance the efficiency of data processing. Industrial networks generate large volumes of data continuously, and the system must be capable of analyzing this data in real time. The use of Autoencoders helps in reducing data complexity, while XGBoost ensures fast and accurate classification.

The project also aims to provide interpretable results, allowing security analysts to understand the detected threats and take appropriate actions. This improves trust, transparency, and accountability in the system.

Overall, the purpose of this project is to create a reliable intrusion detection system that enhances cybersecurity, improves operational stability, and supports the safe functioning of modern industrial systems.

1.3 Objective

The main objective of this project is to design and implement an efficient hybrid intrusion detection system using Autoencoder and XGBoost algorithms. The system aims to provide accurate and scalable detection of cyber threats in industrial communication networks.

The project focuses on identifying vulnerabilities in industrial systems and understanding different types of cyberattacks. It aims to develop a model that can learn complex patterns in network data and accurately distinguish between normal and malicious activities.

Another objective is to improve the detection of unknown and emerging threats. By using deep learning techniques, the system can identify anomalies that do not match existing attack patterns, thereby enhancing security.

The project also aims to ensure scalability by designing a system that can handle large datasets and continuous data streams without performance degradation. Additionally, it seeks to evaluate the system's performance using various metrics such as accuracy, precision, recall, and F1-score.

Furthermore, the project aims to provide interpretable outputs that help users understand the decision-making process of the model. This enhances the usability and reliability of the system in real-world applications.

1.4 Existing System

The existing intrusion detection systems used in industrial environments primarily rely on machine learning and deep learning techniques such as Decision Trees, Support Vector Machines (SVM), Multi-Layer Perceptron (MLP), and basic Autoencoders. These systems are designed to analyze network traffic and detect potential cyber threats.

Most existing systems are trained using publicly available datasets, which provide a limited

representation of real-world industrial scenarios. As a result, these systems often struggle to detect new or unseen attack types. Additionally, many models focus on either anomaly detection or classification, but not both, which limits their effectiveness.

Another limitation of existing systems is their inability to handle large-scale industrial data efficiently. As the volume of data increases, the performance of these systems decreases, leading to slower detection and higher error rates. Furthermore, many models lack interpretability, making it difficult for security analysts to understand the reasons behind detected threats.

Despite these limitations, existing systems have laid the foundation for the use of AI and ML in cybersecurity. However, there is a need for more advanced and scalable solutions that can address the challenges of modern industrial environments.

1.4.1 Disadvantages

The existing intrusion detection systems have several disadvantages that limit their effectiveness in real-world applications. One of the major drawbacks is their dependence on publicly available datasets, which do not accurately represent real industrial environments. This results in poor detection of unknown or emerging threats.

Another disadvantage is the lack of scalability. Many traditional models are not capable of handling large volumes of data generated by industrial networks. This leads to increased processing time and reduced system performance.

High false positive rates are also a common issue in existing systems. This can cause unnecessary alerts and make it difficult for security teams to identify actual threats. Additionally, many systems are not capable of adapting to new attack patterns, making them less effective over time.

Limited interpretability is another drawback. Many machine learning models act as black boxes, providing results without explaining how decisions are made. This reduces trust and makes it difficult to take appropriate action.

Overall, these disadvantages highlight the need for a more advanced intrusion detection system

that is accurate, scalable, and adaptable.

1.5 Proposed System

The proposed system introduces a hybrid intrusion detection model that combines the strengths of Autoencoders and XGBoost to address the limitations of existing systems. This system is designed to provide accurate, scalable, and efficient detection of cyber threats in smart industrial environments.

The Autoencoder component is used for feature extraction and dimensionality reduction. It learns the underlying patterns in the data and generates a compressed representation, which helps in reducing noise and improving data quality. This enables the system to handle complex and high-dimensional data effectively.

The extracted features are then passed to the XGBoost classifier, which performs efficient and accurate classification of network traffic. XGBoost is known for its high performance, speed, and ability to handle large datasets, making it suitable for industrial applications.

The proposed system is capable of detecting both known and unknown attacks by combining anomaly detection and classification techniques. It also supports scalability, allowing it to process large volumes of data in real time.

Additionally, the system provides interpretable outputs, enabling users to understand the detected threats and take appropriate actions. This enhances trust and improves decision-making in cybersecurity operations.

1.5.1 Advantages

The proposed system offers several advantages over existing intrusion detection systems. One of the main advantages is improved detection accuracy. By combining Autoencoders and XGBoost, the system can effectively identify both known and unknown cyber threats.

Another key advantage is scalability. The system is designed to handle large volumes of industrial data efficiently, making it suitable for real-world applications. It can process continuous data streams without significant performance degradation.

The system also provides better adaptability to new and evolving threats. The use of deep learning techniques enables it to learn hidden patterns and detect anomalies that do not match existing attack signatures.

Reduced false positives is another important benefit. By improving feature extraction and classification, the system minimizes incorrect alerts, making it easier for security teams to focus on actual threats.

Additionally, the system offers improved interpretability, allowing users to understand how decisions are made. This enhances trust, transparency, and accountability.

Overall, the proposed system provides a more reliable, efficient, and advanced solution for securing smart industrial environments against cyber threats.

2. LITERATURE SURVEY

• **AI-Driven Smart Industry Systems**

Zhang et al. (2020) explores the integration of artificial intelligence into modern smart manufacturing systems, focusing on automation and intelligent decision-making. The study highlights how AI technologies such as machine learning and deep learning enable industries to analyze large volumes of production data efficiently. AI-driven systems can predict equipment failures, optimize production schedules, and improve quality control. The research emphasizes the role of data-driven approaches in enhancing productivity and reducing operational costs. It also discusses how AI improves system adaptability by learning from dynamic industrial environments. The study further explains the importance of real-time data processing in achieving efficient automation. One of the key contributions of this work is demonstrating how AI can reduce human intervention and improve accuracy in industrial operations. Additionally, the research highlights challenges such as data privacy, security risks, and system integration. This work is highly relevant to intrusion detection systems as it shows how AI can be used for intelligent threat detection. It supports the idea of using hybrid models like Autoencoder and XGBoost for improving system performance. Overall, the study provides a strong foundation for applying AI in industrial cybersecurity.

• **Industrial Artificial Intelligence**

Peres et al. (2020) examines the role of artificial intelligence in enhancing industrial operations through predictive maintenance and automation. The study focuses on how AI models analyze sensor data to detect anomalies and predict failures before they occur. This proactive approach helps industries reduce downtime and maintenance costs. The research highlights the importance of data analytics in improving operational efficiency and system reliability. It also discusses the integration of AI with Industrial IoT systems for continuous monitoring. The study explains how machine learning algorithms can identify hidden patterns in industrial data. Furthermore, it emphasizes the importance of real-time data processing for effective decision-making. One of the key advantages discussed is the ability of AI systems to adapt to changing environments. The research also addresses challenges such as data quality, scalability, and computational complexity. This work is directly related to intrusion detection systems as it demonstrates the use of AI for anomaly detection. It supports the use of Autoencoders for

feature extraction in high-dimensional datasets. Overall, the study highlights the potential of AI in improving both security and efficiency in industrial systems.

• **Cybersecurity in Industry 4.0**

Corallo et al. (2020) provides a comprehensive overview of cybersecurity challenges in Industry 4.0 environments. The study identifies various types of cyber threats, including malware attacks, insider threats, ransomware, and network intrusions. It highlights the increasing vulnerability of industrial systems due to their interconnected nature. The research emphasizes the importance of protecting critical infrastructure and sensitive data. It also discusses different cybersecurity frameworks and risk management strategies. The study explains how traditional security mechanisms are insufficient for modern industrial environments. It suggests the use of advanced techniques such as AI and machine learning for threat detection. The research further highlights the importance of real-time monitoring and intrusion detection systems. One of the key contributions is the classification of industrial assets and associated risks. The study also addresses challenges such as system complexity and scalability. This work strongly supports the need for hybrid intrusion detection models. It provides a clear justification for using advanced techniques like Autoencoder and XGBoost. Overall, the study emphasizes the critical role of cybersecurity in smart industries.

• **Smart Logistics using IoT and Blockchain**

Humayun et al. (2020) explores the integration of IoT and blockchain technologies to enhance security in logistics and supply chain systems. The study explains how IoT devices collect real-time data from various stages of the supply chain. Blockchain technology is used to ensure data integrity and prevent unauthorized modifications. The research highlights the importance of transparency and traceability in logistics systems. It also discusses how these technologies improve trust among stakeholders. The study emphasizes the role of secure communication in industrial environments. It explains how cyberattacks can disrupt supply chains and cause financial losses. The integration of blockchain helps in preventing data tampering and ensuring secure transactions. The research also addresses challenges such as scalability and system complexity. This work is relevant to intrusion detection as it highlights the importance of securing industrial networks. It supports the need for advanced security mechanisms to detect anomalies. Overall, the study provides insights into secure data management in Industry 4.0.

• Fog Computing for Industrial IoT

Pop et al. (2021) introduces fog computing as a solution for processing industrial IoT data closer to the source. The study explains how traditional cloud-based systems face latency issues in real-time applications. Fog computing reduces this latency by performing data processing at the network edge. This improves system performance and enables faster decision-making. The research highlights the importance of real-time data analysis in industrial environments. It also discusses how fog computing enhances scalability and efficiency. The study emphasizes the role of distributed computing in handling large datasets. One of the key advantages is reduced network congestion and improved response time. The research also addresses security challenges in fog-based systems. It suggests integrating intrusion detection mechanisms at the edge. This work is highly relevant to real-time intrusion detection systems. It supports the idea of deploying IDS models closer to data sources. Overall, the study demonstrates the benefits of fog computing in industrial applications.

• Intrusion Detection using XGBoost

Chen et al. (2021) investigates the use of the XGBoost algorithm for detecting network intrusions. The study highlights the advantages of XGBoost, including high accuracy, speed, and scalability. It explains how the algorithm uses gradient boosting to improve classification performance. The research demonstrates that XGBoost performs better than traditional machine learning models. It also shows that the model reduces false positive rates significantly. The study emphasizes the importance of feature selection in improving model performance. It explains how XGBoost handles large datasets efficiently. The research includes experimental results showing high detection accuracy. It also discusses the ability of XGBoost to detect different types of cyberattacks. This work directly supports the use of XGBoost in the proposed system. It validates the choice of XGBoost as a classification algorithm. The study highlights its effectiveness in real-world applications. Overall, it proves that XGBoost is suitable for intrusion detection systems.

• Autoencoder-Based Anomaly Detection

Sakurada & Yairi (2021) explores the use of Autoencoders for detecting anomalies in high-dimensional data. The study explains how Autoencoders learn normal patterns and identify

deviations as anomalies. It highlights the effectiveness of deep learning in feature extraction.

The research demonstrates that Autoencoders can handle complex datasets efficiently. It also explains how dimensionality reduction improves model performance. The study emphasizes the importance of unsupervised learning in anomaly detection. One of the key advantages is the ability to detect unknown attacks. The research includes experimental validation showing improved detection accuracy. It also discusses challenges such as model tuning and computational cost. This work is highly relevant to the proposed system. It supports the use of Autoencoders for feature extraction. The study highlights their role in improving detection of unseen threats. Overall, it provides a strong foundation for hybrid intrusion detection models.

- **Security Challenges in PLC Systems**

Hajda et al. (2021) discusses the security vulnerabilities present in programmable logic controllers (PLCs), which are critical components in industrial control systems. The study highlights how PLCs are often targeted by cyberattacks due to weak authentication mechanisms and outdated firmware. It explains that attackers can exploit these vulnerabilities to manipulate industrial processes. The research emphasizes the importance of securing PLC communication protocols and access control mechanisms. It also discusses different types of attacks such as unauthorized access, malware injection, and command manipulation. The study highlights the need for intrusion detection systems specifically designed for industrial environments. It suggests implementing real-time monitoring and anomaly detection techniques. The research also addresses challenges such as legacy systems and lack of security updates. This work is relevant to the proposed system as it highlights the importance of detecting attacks at the control system level. It supports the use of advanced IDS models to protect critical infrastructure. Overall, the study emphasizes the need for improved cybersecurity measures in industrial systems.

- **Smart Warehouse Architecture**

van Geest et al. (2021) proposes a reference architecture for automated smart warehouses using Industry 4.0 technologies. The study explains how IoT devices, sensors, and automation systems are used for inventory management and tracking. It highlights the importance of real-time data collection and analysis in warehouse operations. The research discusses how automation improves efficiency and reduces human errors. It also emphasizes the need for

secure communication between devices. The study explains how cyberattacks can disrupt warehouse operations and lead to financial losses. It suggests implementing security mechanisms such as encryption and intrusion detection. The research highlights the role of AI in optimizing warehouse processes. It also discusses challenges such as system integration and scalability. This work is relevant to intrusion detection systems as it shows the importance of securing interconnected industrial systems. It supports the need for real-time threat detection. Overall, the study provides insights into secure and efficient warehouse management systems.

• **Big Data and CPS Integration**

Singh (2021) examines the integration of big data analytics with cyber-physical systems in Industry 4.0. The study explains how large volumes of data generated by industrial systems can be analyzed to improve decision-making. It highlights the importance of data-driven approaches in modern industries. The research discusses how big data analytics can detect anomalies and predict system failures. It also emphasizes the need for efficient data processing techniques. The study explains how integrating CPS with big data enhances automation and monitoring. It highlights challenges such as data storage, processing speed, and security. The research suggests using machine learning techniques for analyzing complex datasets. This work is relevant to intrusion detection systems as it supports the use of large datasets for training models. It highlights the importance of scalability in handling industrial data. Overall, the study provides a strong foundation for data-driven intrusion detection systems.

• **Edge Computing for Smart Manufacturing**

Shi et al. (2019) discusses the role of edge computing in improving real-time data processing in smart manufacturing systems. The study explains how processing data at the edge reduces latency compared to cloud-based systems. It highlights the importance of fast response times in industrial environments. The research discusses how edge computing improves system performance and reduces network congestion. It also explains how local data processing enhances security. The study emphasizes the importance of integrating edge computing with IoT systems. It highlights challenges such as resource limitations and system management. The research suggests combining edge computing with AI for better decision-making. This work is relevant to intrusion detection systems as it supports real-time threat detection. It highlights the need for deploying IDS models closer to data sources. Overall, the study demonstrates the

benefits of edge computing in industrial applications.

• **Industry 4.0 State of the Art**

Xu, Xu & Li (2018) provides a comprehensive review of Industry 4.0 technologies and their applications. The study discusses the integration of IoT, AI, and big data in industrial systems. It highlights the transformation of traditional industries into smart systems. The research explains how these technologies improve efficiency and productivity. It also discusses challenges such as system complexity and security risks. The study emphasizes the importance of cybersecurity in Industry 4.0. It suggests implementing advanced security mechanisms to protect industrial systems. The research highlights future trends and research directions. This work is relevant to intrusion detection systems as it identifies key challenges in industrial environments. It supports the need for advanced IDS solutions. Overall, the study provides a broad understanding of Industry 4.0 technologies.

• **IoT-based Smart Manufacturing**

Tao et al. (2018) describes the use of IoT technologies in smart manufacturing systems. The study explains how sensors and devices collect real-time data from industrial processes. It highlights the importance of continuous monitoring and predictive maintenance. The research discusses how IoT improves efficiency and reduces downtime. It also emphasizes the need for secure communication between devices. The study explains how cyber threats can affect IoT systems. It suggests implementing security mechanisms such as encryption and IDS. The research highlights challenges such as scalability and data management. This work is relevant to intrusion detection systems as it emphasizes the need for securing IoT networks. It supports the use of AI-based detection models. Overall, the study provides insights into IoT-enabled industrial systems.

• **Industry 4.0 Technologies Survey**

Lu (2017) provides a detailed survey of key technologies in Industry 4.0. The study discusses IoT, cloud computing, big data, and artificial intelligence. It explains how these technologies enable smart manufacturing. The research highlights the benefits of automation and data-driven decision-making. It also discusses challenges such as security and system integration. The study emphasizes the importance of cybersecurity in industrial environments. It suggests

implementing advanced security mechanisms. This work is relevant to intrusion detection systems as it highlights the need for secure industrial systems. It supports the use of hybrid IDS models. Overall, the study provides a comprehensive overview of Industry 4.0 technologies.

• **Digital Twin Technology**

Tao & Zhang (2017) introduces digital twin technology for monitoring and simulation of industrial systems. The study explains how virtual models replicate physical systems. It highlights the use of digital twins for predictive analysis and optimization. The research discusses how real-time data is used to update digital models. It emphasizes the importance of accurate data for effective simulation. The study also highlights security concerns in digital twin systems. It suggests implementing intrusion detection mechanisms. This work is relevant to intrusion detection systems as it highlights the need for secure monitoring systems. It supports the use of AI-based models for anomaly detection. Overall, the study provides insights into advanced industrial technologies.

• **Machine Learning in Industry 4.0**

Wuest et al. (2016) discusses the application of machine learning in industrial systems. The study explains how ML models are used for predictive maintenance and process optimization. It highlights the importance of data-driven decision-making. The research discusses different ML algorithms and their applications. It also emphasizes challenges such as data quality and model complexity. The study suggests using advanced models for better performance. This work is relevant to intrusion detection systems as it supports the use of ML techniques. It highlights the importance of feature extraction and classification. Overall, the study provides a foundation for applying ML in industrial environments.

• **Big Data for Modern Industry**

Yin & Kaynak (2015) explains the role of big data analytics in modern industrial systems. The study discusses how large datasets are analyzed to improve efficiency. It highlights the importance of data processing and storage techniques. The research explains how big data supports predictive maintenance and optimization. It also discusses challenges such as scalability and security. The study suggests using advanced analytics techniques. This work is

relevant to intrusion detection systems as it supports handling large datasets. It highlights the importance of scalable models. Overall, the study provides insights into big data applications.

• **Cyber-Physical Systems Architecture for Industry 4.0**

Lee et al. (2015) presents a CPS-based architecture integrating physical and digital systems. The study explains how sensors, networks, and computing systems interact. It highlights the importance of real-time data processing. The research discusses how CPS improves automation and efficiency. It also emphasizes security challenges. The study suggests implementing advanced security mechanisms. This work is relevant to intrusion detection systems as it highlights system vulnerabilities. It supports the need for IDS solutions. Overall, the study provides a framework for smart industries.

• **Industrial IoT Architecture**

Xu et al. (2014) explains the architecture of Industrial IoT systems. The study discusses how devices, sensors, and networks are connected. It highlights the importance of real-time communication. The research explains how IIoT improves monitoring and control. It also discusses security challenges. The study suggests implementing secure communication protocols. This work is relevant to intrusion detection systems as it highlights network vulnerabilities. It supports the need for IDS models. Overall, the study provides a foundation for IIoT systems.

• **Cyber-Physical Production Systems**

Monostori (2014) discusses intelligent production systems using cyber-physical technologies. The study explains how automation and data integration improve manufacturing. It highlights the importance of real-time monitoring. The research discusses challenges such as system complexity and security. It suggests using advanced technologies for optimization. This work is relevant to intrusion detection systems as it highlights system vulnerabilities. It supports the need for security mechanisms. Cyber-Physical SystemBaheti & Gill (2011) introduces the concept of cyber-physical systems. The study explains how physical processes are integrated with computational systems. It highlights the importance of communication networks. The research discusses applications in industrial automation. It also emphasizes security challenges. The study suggests implementing advanced security measures.

This work is relevant to intrusion detection systems as it highlights system vulnerabilities. It supports the need for IDS solutions. Overall, the study provides the foundation for Industry 4.0 technologies.

3. SYSTEM REQUIREMENTS

The system requirements define the overall expectations, functionalities, and constraints of the proposed hybrid intrusion detection system. This section provides a clear understanding of what the system is expected to perform and the resources required for its implementation.

The proposed system is designed to detect cyber threats in smart industrial environments using a hybrid approach that combines Autoencoder and XGBoost. The system must be capable of processing large volumes of industrial network data, identifying malicious activities, and providing accurate classification results. It should also ensure scalability, efficiency, and reliability in real-time scenarios.

The system requirements are divided into functional and non-functional requirements, along with a feasibility analysis to evaluate the practicality of the proposed solution.

3.1 Problem Specification

In modern industrial environments, the increasing integration of digital technologies has created complex communication networks that are highly vulnerable to cyber threats. Traditional intrusion detection systems are not capable of effectively handling these threats due to their limited ability to detect unknown attacks and process large-scale data.

The main problem addressed in this project is the inefficiency of existing intrusion detection systems in detecting sophisticated and evolving cyberattacks in industrial networks. Most existing systems rely on static datasets and predefined rules, which limits their ability to adapt to new attack patterns. Additionally, they struggle with scalability when dealing with high-volume industrial data streams.

Another major issue is the high false positive rate, which leads to unnecessary alerts and reduces the effectiveness of security monitoring. This creates additional workload for security analysts and increases the chances of overlooking real threats.

Furthermore, industrial systems require real-time detection capabilities, but many existing models are not optimized for fast processing. This delay in detection can result in severe consequences, including system failures and financial losses.

To address these challenges, the problem is defined as developing a hybrid intrusion detection system that can:

- Detect both known and unknown cyber threats
- Handle large volumes of industrial data efficiently
- Reduce false positives and improve accuracy
- Provide scalable and real-time performance

The proposed solution aims to overcome these limitations by integrating deep learning and machine learning techniques.

3.2 Modules and Their Functionalities

The proposed system is divided into several modules, each responsible for a specific functionality. These modules work together to ensure efficient intrusion detection and system performance.

The first module is the Data Collection Module, which gathers network traffic data from industrial communication systems. This data may include normal and malicious traffic, which is used for training and testing the model.

The second module is the Data Preprocessing Module, which prepares the collected data for analysis. This includes removing noise, handling missing values, normalizing features, and converting raw data into a suitable format for model input.

The third module is the Feature Extraction Module (Autoencoder). This module uses an Autoencoder to learn hidden patterns in the data and generate a compressed representation known as latent features. This helps in reducing dimensionality and improving data quality.

The fourth module is the Classification Module (XGBoost). This module uses the extracted features to classify network traffic into normal or malicious categories. XGBoost is chosen for its high accuracy, speed, and ability to handle large datasets.

The fifth module is the Detection Module, which identifies different types of cyberattacks such as DoS, ransomware, insider attacks, and APTs based on classification results.

The sixth module is the Alert and Reporting Module, which generates alerts when suspicious activity is detected. It also provides detailed reports for analysis and decision-making.

Finally, the Evaluation Module measures the performance of the system using metrics such as accuracy, precision, recall, and F1-score.

3.3 Functional Requirements

Functional requirements define the specific operations that the system must perform. These requirements describe how the system should behave and interact with data.

The system must be capable of collecting and processing industrial network data from various sources. It should support both real-time and batch data processing to ensure flexibility in different deployment scenarios.

The system must perform data preprocessing, including cleaning, normalization, and transformation of input data. This ensures that the data is suitable for analysis and improves the performance of the model.

The system must implement an Autoencoder for feature extraction. It should be able to learn meaningful patterns and reduce the dimensionality of the data without losing important information.

The system must implement XGBoost for classification. It should accurately classify network traffic into normal and malicious categories and identify different types of cyberattacks.

The system must detect anomalies and unknown attack patterns using learned representations from the Autoencoder. This improves the system's ability to handle emerging threats.

The system must generate alerts when suspicious activity is detected. These alerts should include relevant information such as type of attack, severity, and affected components.

The system must provide performance evaluation metrics such as accuracy, precision, recall, and F1-score to assess its effectiveness.

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints of the system. These requirements ensure that the system performs efficiently and reliably under different conditions.

The system must provide high performance, ensuring fast processing of large volumes of data. It should be capable of real-time detection without significant delays.

The system must be scalable, allowing it to handle increasing data volumes and network complexity. It should perform efficiently even as the size of industrial networks grows.

The system must ensure reliability, providing consistent and accurate results under different conditions. It should minimize errors and maintain stable performance.

The system must be secure, protecting sensitive data and preventing unauthorized access. It should follow best practices in cybersecurity.

The system must be user-friendly, providing clear and understandable outputs. The alert and reporting module should present information in an easy-to-interpret format.

The system must be maintainable, allowing easy updates and modifications. This is important for adapting to new threats and improving system performance.

The system must provide interpretability, enabling users to understand how decisions are made. This enhances trust and transparency.

3.5 Feasibility Report

The feasibility analysis evaluates whether the proposed system is practical and achievable within given constraints. It includes technical, economic, and operational feasibility.

From a technical feasibility perspective, the project is achievable using existing tools and technologies such as Python, machine learning libraries, and datasets. The use of Autoencoders and XGBoost is well-supported, and sufficient resources are available for implementation.

From an economic feasibility perspective, the project is cost-effective. Most of the required tools and frameworks are open-source, reducing development costs. The system does not require expensive hardware, making it suitable for academic and industrial applications.

From an operational feasibility perspective, the system is easy to use and can be integrated into existing industrial environments. It provides clear outputs and alerts, making it useful for security analysts.

The project is also feasible in terms of time, as it can be completed within the given academic schedule. Proper planning and modular development ensure timely completion.

Overall, the proposed system is feasible and practical, offering a reliable solution for improving cybersecurity in smart industrial environments.

4. SOFTWARE AND HARDWARE REQUIREMENT ANALYSIS

The software and hardware requirement analysis defines the resources needed to design, develop, and deploy the proposed hybrid intrusion detection system. This section ensures that all necessary tools, technologies, and system configurations are clearly identified before implementation.

The proposed system relies on machine learning and deep learning techniques, which require both efficient software frameworks and adequate hardware resources. Proper selection of these requirements is essential to ensure smooth system performance, scalability, and reliability. The system must be capable of processing large volumes of industrial network data while maintaining accuracy and speed.

This section is divided into system specification, software requirements, and hardware requirements, each providing detailed insights into the resources required for successful implementation.

4.1 System Specification

The system specification describes the overall configuration and environment required to develop and run the proposed intrusion detection system. It includes details about the operating system, programming environment, tools, and system architecture.

The proposed system is designed to operate on a standard computing platform that supports machine learning and data processing tasks. It requires an operating system such as Windows, Linux, or macOS, with Linux being preferred due to its compatibility with cybersecurity tools and machine learning frameworks.

The system is developed using Python, which provides extensive support for machine learning and data analysis. Libraries such as NumPy, Pandas, Scikit-learn, TensorFlow or PyTorch, and XGBoost are used to implement the hybrid model. These libraries enable efficient data processing, feature extraction, and model training.

The development environment may include tools such as Jupyter Notebook, VS Code, or

PyCharm, which provide an interactive interface for coding, debugging, and visualization. Visualization libraries such as Matplotlib and Seaborn are used to analyze data and present results.

The system follows a modular architecture consisting of data collection, preprocessing, feature extraction, classification, and reporting modules. Each module is designed to perform specific tasks, ensuring flexibility and ease of maintenance.

The system also requires access to datasets for training and testing. These datasets may include industrial network traffic data containing both normal and malicious activities. The system should be capable of handling structured and unstructured data formats.

Overall, the system specification ensures that the proposed solution is compatible with modern computing environments and capable of supporting advanced intrusion detection techniques.

4.2 Software Requirements

The software requirements define the tools, frameworks, and technologies needed to implement the proposed system. These requirements are essential for developing, training, and evaluating the hybrid intrusion detection model.

The primary programming language used in this project is Python due to its simplicity, flexibility, and extensive support for machine learning and cybersecurity applications. Python provides a wide range of libraries that simplify data processing and model development.

Key libraries used in the project include NumPy and Pandas for data manipulation and preprocessing. These libraries help in handling large datasets, cleaning data, and performing transformations आवश्यक for model training.

Scikit-learn is used for implementing basic machine learning functions, data splitting, and performance evaluation. It provides tools for calculating metrics such as accuracy, precision, recall, and F1-score.

Deep learning frameworks such as TensorFlow or PyTorch are used to implement the Autoencoder model. These frameworks enable the system to learn complex patterns and

perform feature extraction efficiently.

XGBoost is used as the classification algorithm due to its high performance and ability to handle large datasets. It provides fast and accurate classification results, making it suitable for intrusion detection tasks.

Visualization tools such as Matplotlib and Seaborn are used to analyze data and display results in graphical form. These tools help in understanding model performance and identifying patterns in the data.

Additionally, tools like Jupyter Notebook or VS Code are used as development environments, providing an interactive platform for coding and testing.

Overall, the software requirements ensure that the system is built using reliable, efficient, and widely supported technologies.

4.3 Hardware Requirements

The hardware requirements define the physical components needed to develop and run the proposed system. Since the project involves machine learning and data processing, the hardware must be capable of handling computational tasks efficiently.

A standard computer system with a modern processor such as Intel Core i5/i7 or equivalent is sufficient for development and testing. A minimum of 8 GB RAM is required, although 16 GB RAM is recommended for better performance when handling large datasets.

The system should have adequate storage capacity, preferably at least 256 GB of SSD storage, to store datasets, models, and project files. SSD storage is preferred over HDD as it provides faster data access and improves overall system performance.

A GPU is optional but recommended for faster training of deep learning models. GPUs significantly reduce training time for Autoencoders and improve efficiency when working with large datasets.

The system should also have a stable network connection if real-time data collection or cloud-based tools are used. However, the project can also be executed in an offline environment using pre-collected datasets.

Input and output devices such as a keyboard, mouse, and display monitor are required for user interaction. These components ensure smooth operation and usability of the system.

Overall, the hardware requirements are moderate and can be fulfilled using commonly available computing systems, making the project feasible for academic and practical implementation.

5. SYSTEM DESIGN

5.1 System Architecture

Industrial Communication Networks (ICNs) serve as the foundation for intelligent automation in the smart industrial era and enable continuous interaction among robots, sensors, human-machine interfaces (HMI), instruments, monitoring tools, and control systems. These interconnected components generate large volumes of operational data that flow into data collection and preprocessing units, where Autoencoders are applied to normalize, compress, and refine real-world industrial data.

However, the interconnected nature of ICNs also exposes industries to several cybersecurity intrusions such as insider threats, advanced persistent threats (APT), ransomware, and denial-of-service (DoS) attacks. These intrusions can severely disrupt operations, compromise sensitive information, and interrupt critical processes such as robotics control, supply chain management, quality assurance, energy management, environmental monitoring, logistics, agriculture automation, fleet management, and healthcare equipment monitoring.

In existing systems, AI-based intrusion detection relies on models like Multi-Layer Perceptron (MLP), Autoencoders, and Decision Trees for identifying blackhole, grayhole, flooding, and scheduling attacks. However, these models depend predominantly on publicly available datasets, limiting their effectiveness in detecting unseen or real-world industrial attack patterns and reducing scalability when working with large industrial datasets.

To overcome these challenges, the proposed system integrates Autoencoders with the XGBoost classifier, forming a hybrid intrusion detection model capable of processing real industrial data efficiently. This hybrid approach enhances detection accuracy, scalability, robustness, and real-time response. It also improves trust and accountability in incident response by providing more transparent and reliable intrusion detection suitable for modern industrial environments.

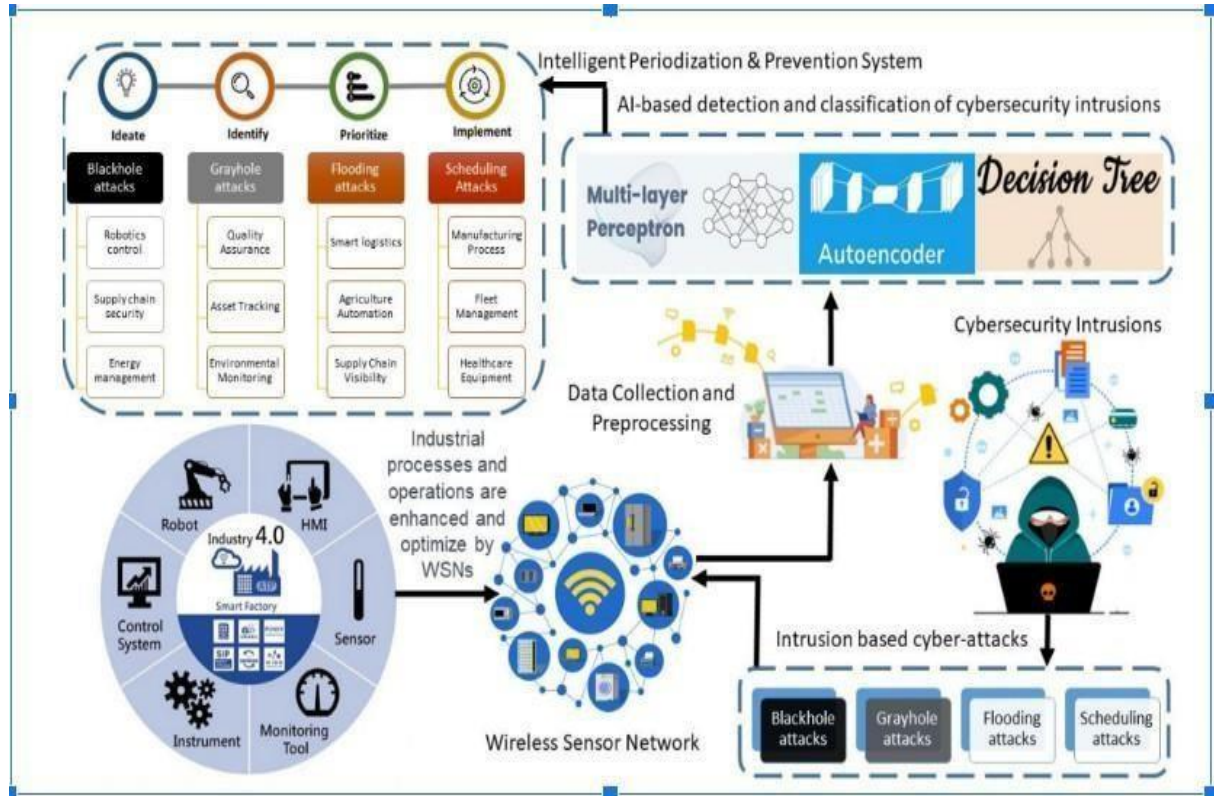


FIGURE 5.1.1. Proposed Model for Smart Industrial Era

To mitigate cybersecurity risks, the system introduces a hybrid AI-based intrusion detection and classification mechanism. In this architecture, the Autoencoder performs deep preprocessing by extracting meaningful latent features from high-volume industrial data, reducing noise and dimensionality. These refined features are then passed to the XGBoost classifier, which performs efficient and scalable classification to detect both known and previously unseen cyberattack patterns.

This hybrid combination enables intelligent analysis of complex intrusion behaviors and significantly strengthens detection accuracy and robustness. Once cyber threats are identified, an Intelligent Prioritization and Response System evaluates the severity of each intrusion. Critical threats are prioritized and addressed immediately to prevent operational disruptions.

The response mechanism follows a structured workflow consisting of Ideate, Identify, Prioritize, and Implement phases.

5.2 Data flow Diagram

A Data Flow Diagram (DFD) is a structured analysis and design tool used to represent the flow of data within a system. It visually illustrates how data enters the system, how it is processed, stored, and how it moves between different components. The DFD breaks down a system into processes, data stores, data flows, and external entities, showing both the logical and physical aspects of the data movement.

External Entities represent outside sources or destinations of data such as users, sensors, industrial devices, or external systems. These entities provide input data to the system or receive processed output.

Processes represent the core operations performed by the system, such as data preprocessing, feature extraction using Autoencoder, classification using XGBoost, and intrusion detection. These processes transform input data into meaningful outputs.

Data Stores represent repositories where data is stored for future use. This includes datasets, processed data, trained models, and logs generated during system operation.

Data Flows represent the movement of data between different components of the system. They show how data travels from input sources through processing modules and finally to output stages such as alerts and reports.

The DFD provides a clear and simplified view of the system without focusing on implementation details. It helps stakeholders understand how data is captured, processed, and delivered across the system. Typically, Level 0 (context diagram) provides an overall view of the system, while Level 1 and Level 2 diagrams break down the processes into more detailed components.

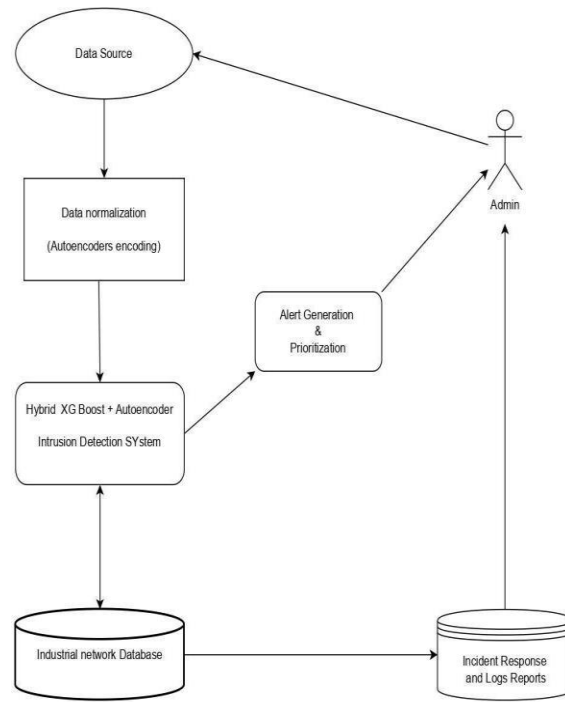


FIGURE 5.2.1. Data Flow Diagram of Intrusion Detection System

6. UML DIAGRAMS

Unified Modeling Language (UML) is used to represent the design and functionality of a system using graphical notations. It helps in understanding system behavior, interaction between components, and internal structure. UML diagrams improve clarity, communication, and system design efficiency.

6.1 Use Case Diagram

The use case diagram represents the high-level functionality of the proposed Hybrid Intrusion Detection System for Smart Industrial Networks. It illustrates how different users (actors) interact with the system and what operations are performed within the system boundary. The diagram helps in understanding the overall workflow and responsibilities of the system in a simplified graphical form.

In this diagram, the system is represented as a central block containing multiple use cases such as Collects Data, Data Normalization, Training & Update Model, Intrusion Detection, Alert Generation & Threat Prioritization, and Incident Logs & Report. These use cases define the major functionalities of the system. The actors, typically Admin and System Operator, interact with these functionalities to monitor and manage cybersecurity operations.

The process begins with the Collects Data use case. In this stage, the system gathers industrial network data from various sources such as sensors, machines, and communication networks. This data includes both normal operational data and potentially malicious traffic. The data collection process is continuous and forms the foundation for all subsequent operations in the system.

Once the data is collected, it moves to the Data Normalization phase. In this stage, the raw data is preprocessed to remove noise, handle missing values, and convert it into a structured format suitable for machine learning models. Normalization ensures that the data is consistent and improves the performance of the detection model.

The next use case is Training & Update Model, where the system trains the hybrid model using Autoencoder and XGBoost. The Autoencoder extracts meaningful features from the data, while

XGBoost performs classification. This module also updates the model periodically to adapt to new patterns and evolving cyber threats, ensuring that the system remains effective over time.

After training, the system performs Intrusion Detection, which is the core functionality of the model. In this stage, incoming network data is analyzed to identify whether it is normal or malicious. The system detects various types of cyberattacks such as Denial of Service (DoS), ransomware, insider threats, and advanced persistent threats (APT). This process is crucial for maintaining the security of industrial networks.

Once an intrusion is detected, the system proceeds to Alert Generation & Threat Prioritization. In this phase, alerts are generated to notify the admin or system operator about potential threats. The system also prioritizes threats based on their severity level, allowing critical threats to be addressed immediately. This ensures efficient incident response and resource allocation.

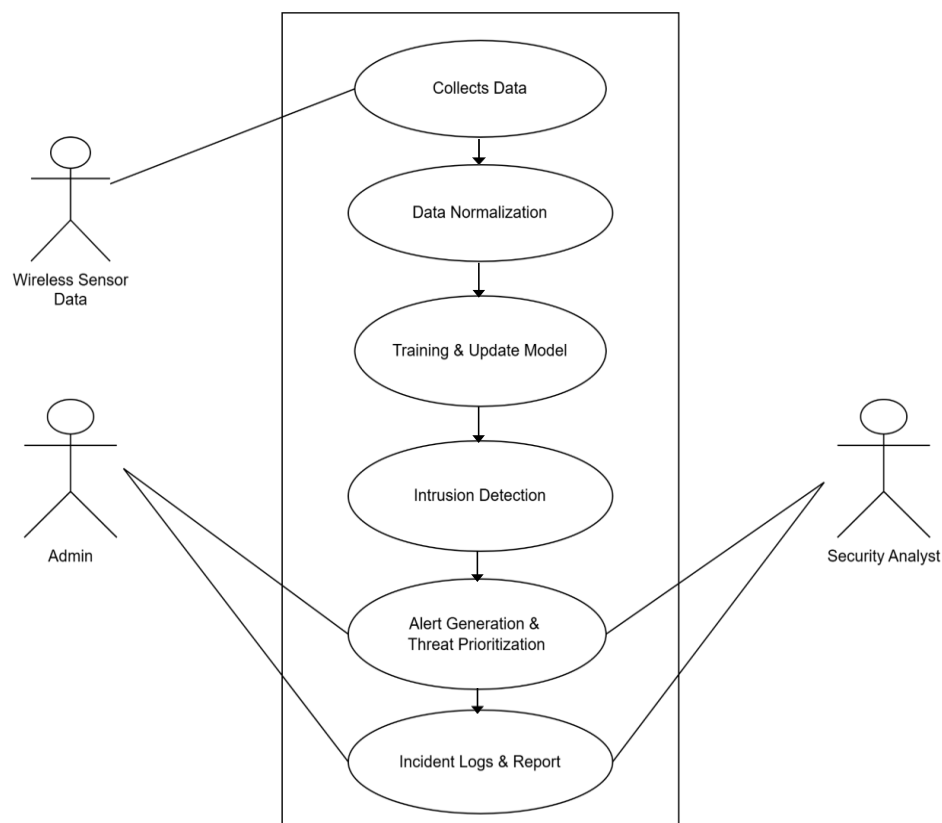


FIGURE 6.1.1 Use Case Diagram

6.2 Activity Diagram

The activity diagram represents the step-by-step workflow of the proposed Hybrid Intrusion Detection System using Autoencoder and XGBoost. It provides a detailed view of how data flows through the system, how decisions are made, and how different processes are executed to detect and respond to cyber threats in smart industrial environments. This diagram is useful for understanding the dynamic behavior of the system and how it operates in real-time conditions.

The process begins with the initial node, which indicates the start of the system operation. The first activity is “collects industrial network data”, where the system gathers data from industrial communication networks. This data may include information from sensors, machines, controllers, and network traffic. The collected data can consist of both normal operational data and potentially malicious activities.

The next step is “Normalizes raw data using autoencoders”. In this stage, the system preprocesses the collected data to make it suitable for analysis. Autoencoders are used to extract meaningful features from high-dimensional data by reducing noise and compressing the data into a lower-dimensional representation. This improves the quality of input data and enhances the performance of the classification model.

After preprocessing, the data is passed to the “Intrusion Detection System (XGBoost + Autoencoder)”. This is the core component of the system, where the hybrid model analyzes the data. The Autoencoder provides refined feature representations, while XGBoost performs classification to determine whether the data represents normal behavior or a cyberattack. This combination improves detection accuracy and helps identify both known and unknown threats.

The system then reaches a decision point labeled “Attack detected?”. This is a critical step where the system determines whether the analyzed data indicates the presence of an attack.

If the decision is “No”, the system proceeds to the “Continue monitoring” activity. In this case, the system keeps observing incoming data and repeats the process continuously. This loop ensures that the system operates in real-time and continuously monitors the industrial network for potential threats.

If the decision is “Yes”, the system follows the attack handling path. The first step in this path is “Generate alerts & Prioritize threats”. Here, the system generates alerts to notify administrators about detected threats. It also prioritizes these threats based on severity, ensuring that critical attacks are addressed first.

Next, the system performs “Store incident logs in database”. In this stage, detailed information about the detected attack is recorded, including the type of attack, time of occurrence, and system response. This data is useful for future analysis, auditing, and improving the detection model.

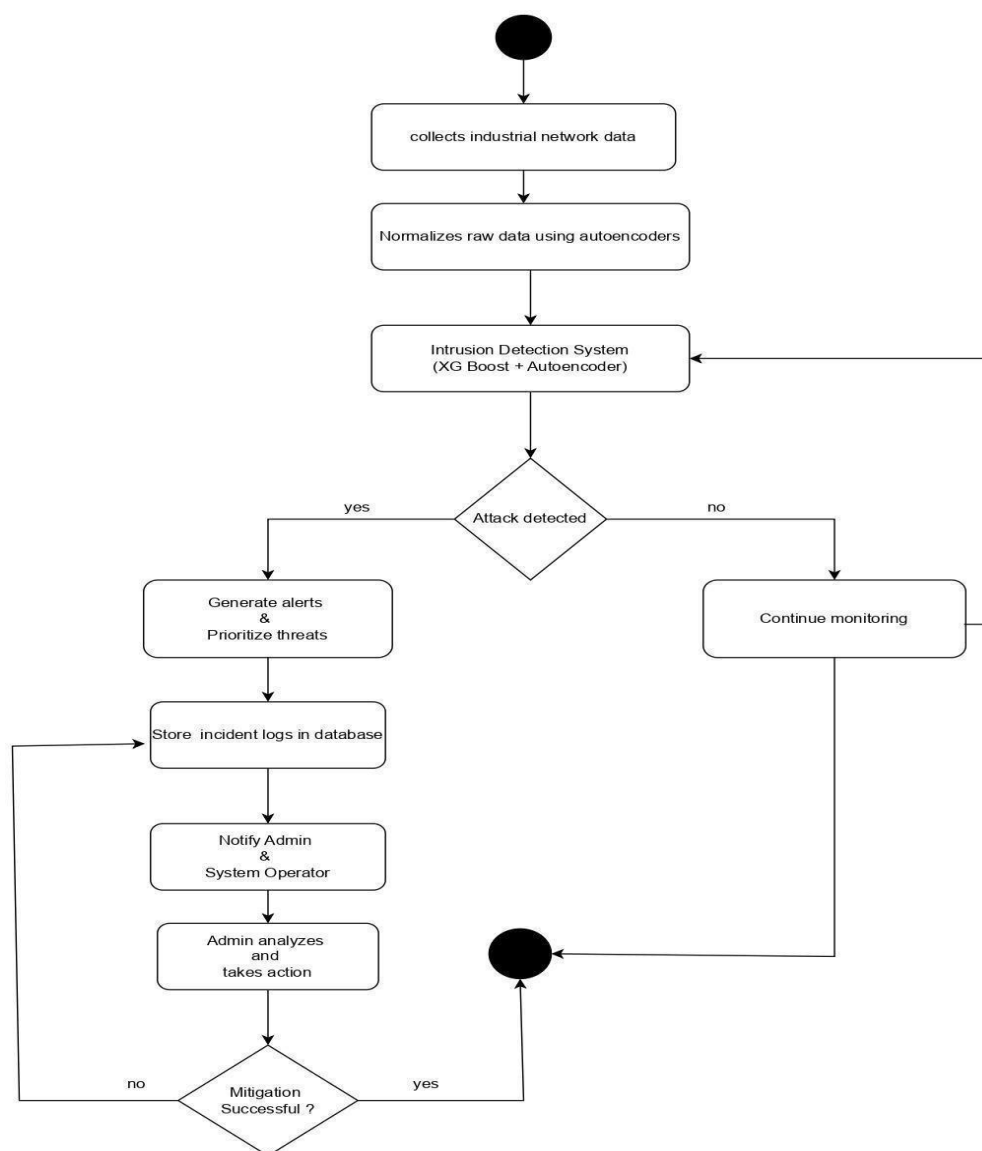


FIGURE 6.2.1 Activity Diagram

6.3 Sequence Diagram

The sequence diagram illustrates the interaction between different components of the Hybrid Intrusion Detection System over time. It shows how messages are exchanged between system modules and how the overall intrusion detection process is executed step-by-step. This diagram is particularly useful for understanding the dynamic behavior of the system and the order in which operations are performed.

In this diagram, the main participants include the Admin (actor), Industrial Network Data module, Data Normalization module, Intrusion Detection System, Alert Generation & Threat Prioritization module, and Incident Logs & Report Database. Each participant is represented by a vertical lifeline, and interactions between them are shown using horizontal arrows.

The sequence begins with the Admin or system initiating the process, where industrial network data is generated or collected from various devices such as sensors, controllers, and communication networks. This data is then passed to the next component in the sequence.

The first interaction is “Sends data()”, where the Industrial Network Data module sends raw data to the Data Normalization module. This step is essential because raw industrial data is often unstructured and contains noise. Proper preprocessing is required before it can be analyzed by machine learning models.

In the next step, the Data Normalization module performs “Normalize and encode data()”. Here, the system cleans the data, removes inconsistencies, handles missing values, and converts it into a structured format. Autoencoders may be used in this stage to extract meaningful features and reduce dimensionality. The processed data is then forwarded to the Intrusion Detection System.

The Intrusion Detection System is the core component of the model, where the hybrid approach (Autoencoder + XGBoost) is applied. The system analyzes the incoming data and performs classification. This interaction is represented as “Detects suspicious intrusions()”, where the model determines whether the data represents normal behavior or a potential cyberattack.

If suspicious activity is detected, the system sends the results to the Alert Generation & Threat Prioritization module. This module is responsible for generating alerts and ranking threats

based on their severity. The interaction “Generate alerts and provide ranking()” indicates that the system not only detects attacks but also prioritizes them to ensure that critical threats are addressed first.

Simultaneously, the system performs “notify threats()”, which sends notifications back to the Admin. This ensures that the administrator is informed about detected threats in real time. Notifications may include alert messages, severity levels, and recommended actions.

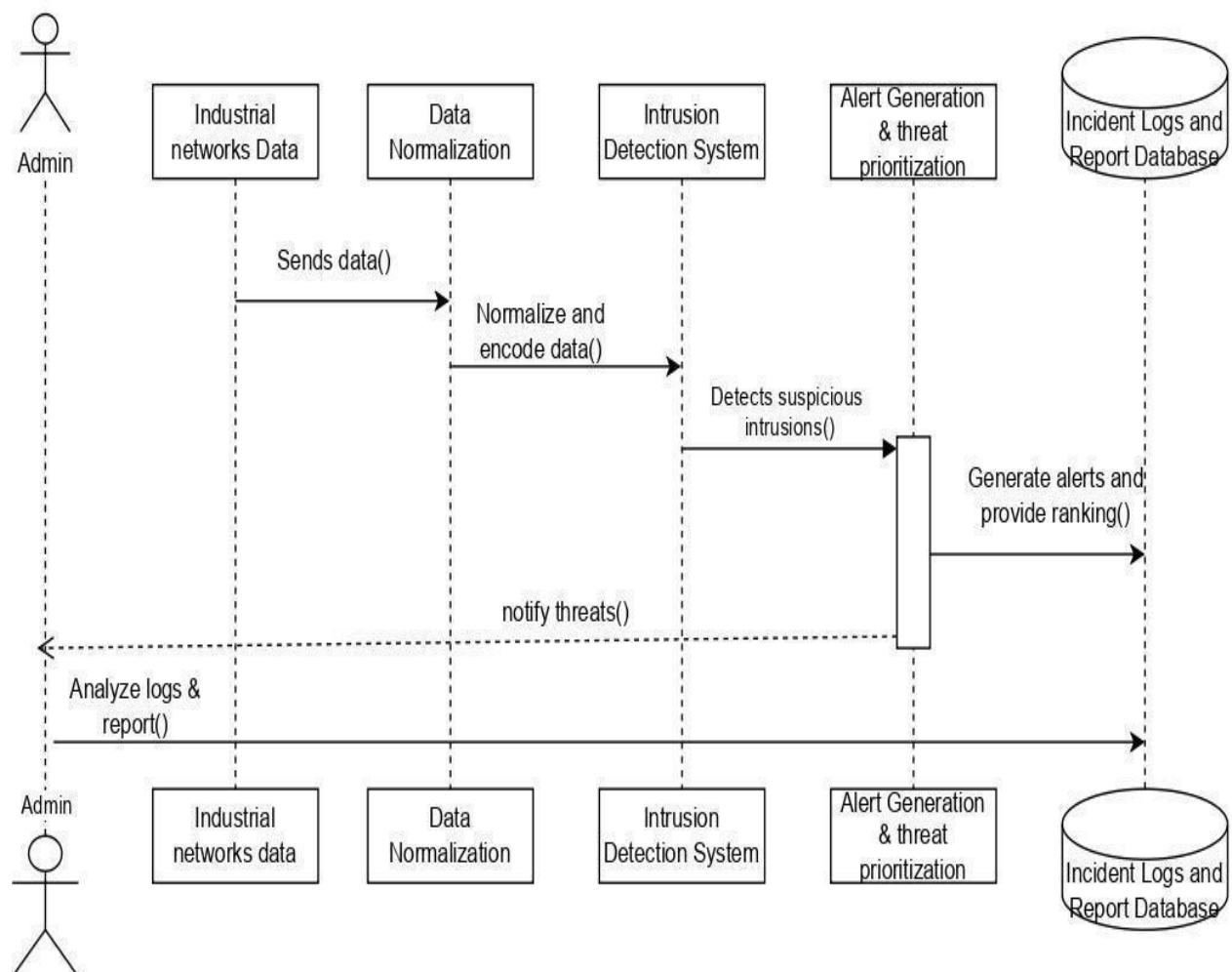


FIGURE 6.3.1 Sequence Diagram

6.4 Class Diagram

The class diagram represents the static structure of the Hybrid Intrusion Detection System, showing the different classes, their attributes, methods, and relationships. It provides a detailed blueprint of how data and functionalities are organized within the system. This diagram is important for understanding how different components interact internally and how the system is designed using object-oriented principles.

In this diagram, the main classes include Industrial Networks Data, Data Normalization, Intrusion Detection System, Alert Generation & Threat Prioritization, Incident Logs & Report, Admin, and System Operator. Each class represents a specific module of the system with defined attributes and operations.

The Industrial Networks Data class is responsible for collecting and sending network data from industrial devices. It contains attributes such as network device ID, timestamp, signal strength, and packet rate, which represent the characteristics of network traffic. The methods `collectData()` and `sendData()` are used to gather and forward data to the next stage. This class acts as the entry point of the system.

The next class is Data Normalization, which handles preprocessing and feature extraction. It contains attributes such as encoder weights and normalization rules, which are used to transform raw data into a structured format. The methods `PreprocessData()` and `generateFeatures()` are responsible for cleaning the data and extracting meaningful features using Autoencoders. This class plays a crucial role in improving data quality and preparing it for analysis.

The processed data is then passed to the Intrusion Detection System class, which is the core component of the model. This class contains attributes such as training set and testing set, which are used for model training and evaluation. The methods `trainModel()`, `testModel()`, and `detectAttack()` define the operations of the hybrid model. The system uses Autoencoder for feature extraction and XGBoost for classification to detect cyber threats. This class is responsible for identifying whether the incoming data represents normal activity or an attack.

Once an intrusion is detected, the system moves to the Alert Generation & Threat Prioritization

class. This class contains attributes such as alert ID, severity, and timestamp, which are used to store information about detected threats. The methods `generateAlerts()` and `prioritizeThreats()` are responsible for creating alerts and ranking threats based on their importance. This ensures that critical threats are handled first.

The Incident Logs & Report class is responsible for storing and managing all detected events. It includes attributes such as log ID, attack type, and action taken. The methods `saveLogs()` and `generateReport()` are used to record incidents and generate detailed reports. This class is essential for auditing, analysis, and improving the system over time.

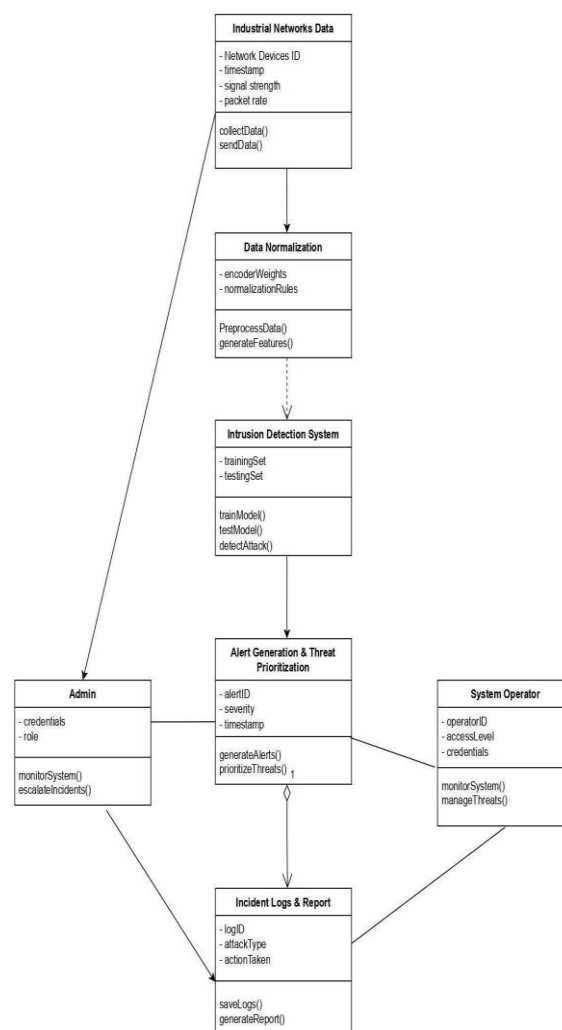


FIGURE 6.4.1 Class Diagram

7. CODE AND IMPLEMENTATION

```
import os

# Suppress TensorFlow GPU warnings
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'


from tkinter import *

from tkinter import filedialog

import tkinter

import matplotlib.pyplot as plt

import numpy as np

import pandas as pd


from sklearn.model_selection import train_test_split

from sklearn import svm

from sklearn.metrics import accuracy_score

from sklearn.feature_selection import SelectKBest, chi2

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import RandomForestClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.preprocessing import MinMaxScaler


from keras.models import Sequential

from keras.layers import Dense
```

```

main = tkinter.Tk()

main.title("Network Intrusion Detection")

main.geometry("1300x1200")


# --- Global Variables ---

filename = ""

X = Y = X_train = X_test = y_train = y_test = None

svm_acc = ann_acc = LR_acc = DT_acc = RFT_acc = 0

classifier = None

selector_svm = None


def isfloat(value):

    try:

        float(value)

        return True

    except:

        return False


def splitdataset(balance_data):

    # Use .iloc to avoid position-based warnings

    X_raw = balance_data.iloc[:, 0:38].values

    Y_raw = balance_data.iloc[:, 38].values


    scaler = MinMaxScaler()

```

```
X_scaled = scaler.fit_transform(X_raw)
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X_scaled, Y_raw, test_size=0.2, random_state=0
```

```
)
```

```
return X_scaled, Y_raw, X_train, X_test, y_train, y_test
```

```
def upload():
```

```
    global filename
```

```
    text.delete('1.0', END)
```

```
    filename = filedialog.askopenfilename(initialdir=".")
```

```
    pathlabel.config(text=filename)
```

```
    text.insert(END, "Dataset Loaded\n\n")
```

```
def preprocess():
```

```
    global filename
```

```
    text.delete('1.0', END)
```

```
    if not filename:
```

```
        text.insert(END, "Please upload a dataset first.\n")
```

```
        return
```

```
    try:
```

```
        df = pd.read_csv(filename)
```

```

# Display Info Headers immediately

text.insert(END, "Removed non numeric characters from dataset and saved inside
clean.txt file\n\n")

text.insert(END, "Dataset Information\n\n")

text.update()


# Vectorized preprocessing

processed_df = df.iloc[:, 0:41].apply(pd.to_numeric, errors='coerce').fillna(0)

labels = df.iloc[:, 41].astype(str).str.lower().apply(lambda x: 0 if x == 'normal' else 1)

processed_df['label'] = labels


# Save to file

processed_df.to_csv("clean.txt", index=False, header=False)


# Show the processed row samples (Top 15 rows)

sample_data = processed_df.head(15).astype(str).values

for row in sample_data:

    line = ",".join(row) + "\n"

    text.insert(END, line)

text.update()


except Exception as e:

    text.insert(END, f"Error during preprocessing: {e}\n")

```



```

def generateModel():

    global X, Y, X_train, X_test, y_train, y_test

    text.delete('1.0', END)

    if not os.path.exists("clean.txt"):

        text.insert(END, "Cleaned file not found. Please preprocess first.\n")

        return

    try:

        balance_data = pd.read_csv("clean.txt", header=None)

        X, Y, X_train, X_test, y_train, y_test = splitdataset(balance_data)

        text.insert(END, "Train & Test Model Generated\n\n")

        text.insert(END, f"Total Dataset Size : {len(balance_data)}\n")

        text.insert(END, f"Training Size : {len(X_train)}\n")

        text.insert(END, f"Testing Size : {len(X_test)}\n")

    except Exception as e:

        text.insert(END, f"Error: {e}\n")


def runSVM():

    global svm_acc, classifier, selector_svm

    text.delete('1.0', END)

    if X_train is None:

        text.insert(END, "Generate Training Model first.\n")

        return

```

```

selector_svm = SelectKBest(score_func=chi2, k=15)

X_train1 = selector_svm.fit_transform(X_train, y_train.astype('int'))

X_test1 = selector_svm.transform(X_test)


model = svm.SVC(kernel='rbf', probability=True)

model.fit(X_train1, y_train.astype('int'))

pred = model.predict(X_test1)

svm_acc = accuracy_score(y_test.astype('int'), pred) * 100

classifier = model

text.insert(END, f"SVM Accuracy : {svm_acc:.2f}%\n\n")

```

```

def runDT():

    global DT_acc

    text.delete('1.0', END)

    if X_train is None:

        text.insert(END, "Generate Training Model first.\n")

    return

```

```

selector = SelectKBest(score_func=chi2, k=15)

X_train1 = selector.fit_transform(X_train, y_train.astype('int'))

X_test1 = selector.transform(X_test)

model = DecisionTreeClassifier()

model.fit(X_train1, y_train.astype('int'))

pred = model.predict(X_test1)

```

```

DT_acc = accuracy_score(y_test.astype('int'), pred) * 100

text.insert(END, f"Decision Tree Accuracy : {DT_acc:.2f}%\n\n")


def runRFT():

    global RFT_acc

    text.delete('1.0', END)

    if X_train is None:

        text.insert(END, "Generate Training Model first.\n")

        return


    selector = SelectKBest(score_func=chi2, k=15)

    X_train1 = selector.fit_transform(X_train, y_train.astype('int'))

    X_test1 = selector.transform(X_test)

    model = RandomForestClassifier()

    model.fit(X_train1, y_train.astype('int'))

    pred = model.predict(X_test1)

    RFT_acc = accuracy_score(y_test.astype('int'), pred) * 100

    text.insert(END, f"Random Forest Accuracy : {RFT_acc:.2f}%\n\n")


def runLR():

    global LR_acc

    text.delete('1.0', END)

    if X_train is None:

        text.insert(END, "Generate Training Model first.\n")

```

```

return

selector = SelectKBest(score_func=chi2, k=15)

X_train1 = selector.fit_transform(X_train, y_train.astype('int'))

X_test1 = selector.transform(X_test)

model = LogisticRegression(max_iter=500)

model.fit(X_train1, y_train.astype('int'))

pred = model.predict(X_test1)

LR_acc = accuracy_score(y_test.astype('int'), pred) * 100

text.insert(END, f"Logistic Regression Accuracy : {LR_acc:.2f}%\n\n")


def runANN():

    global ann_acc

    text.delete('1.0', END)

    if X_train is None:

        text.insert(END, "Generate Training Model first.\n")

        return

    selector = SelectKBest(score_func=chi2, k=25)

    X_train1 = selector.fit_transform(X_train, y_train.astype('int'))

    X_test1 = selector.transform(X_test)


    model = Sequential()

    model.add(Dense(30, input_dim=25, activation='relu'))

```

```

model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

model.fit(X_train1, y_train.astype('int'), epochs=10, batch_size=32, verbose=0)


loss, accuracy = model.evaluate(X_test1, y_test.astype('int'), verbose=0)

ann_acc = accuracy * 100

text.insert(END, f'ANN Accuracy : {ann_acc:.2f}%\n\n')

```

```
def detectAttack():
```

```

    global classifier, selector_svm

    text.delete('1.0', END)

    if classifier is None or selector_svm is None:

        text.insert(END, "Please run SVM first to train the reference classifier.\n")

    return

```

```
file = filedialog.askopenfilename(initialdir=".")
```

```
if file:
```

```

    try:

        text.insert(END, "Test Data Uploaded\n\n")

        text.update()

```

```
# Fix for 'duration' error: Handle Headers and non-numeric strings
```

```
test_df = pd.read_csv(file)
```

```
# If the first value of first column is NOT a number, it's a header
```

```

if not isfloat(str(test_df.iloc[0,0])):

    # Already read correctly with header skip

    pass

elif isfloat(test_df.columns[0]):

    # If column name is a number, file had no header, re-read without skipping

    test_df = pd.read_csv(file, header=None)


# Clean and Select features

test_data = test_df.iloc[:, 0:38].apply(pd.to_numeric, errors='coerce').fillna(0).values

scaler = MinMaxScaler()

test_data_scaled = scaler.fit_transform(test_data)

test_selected = selector_svm.transform(test_data_scaled)


y_pred = classifier.predict(test_selected)

for i, res in enumerate(y_pred):

    msg = f"Row {i+1}: Detected Attack\n" if res == 1 else f"Row {i+1}: Normal
Traffic\n"

    text.insert(END, msg)

except Exception as e:

    text.insert(END, f"Error during detection: {e}\n")


def graph():

    height = [svm_acc, ann_acc, LR_acc, DT_acc, RFT_acc]

    bars = ('SVM', 'ANN', 'LR', 'DT', 'RFT')

```

```

y_pos = np.arange(len(bars))

test_data = test_df.iloc[:, 0:38].apply(pd.to_numeric, errors='coerce').fillna(0).values

scaler = MinMaxScaler()

global classifier, selector_svm

text.delete('1.0', END)

if classifier is None or selector_svm is None:

    text.insert(END, "Please run SVM first to train the reference classifier.\n")

    return

y_pred = classifier.predict(test_selected)

for i, res in enumerate(y_pred):

    msg = f"Row {i+1}: Detected Attack\n" if res == 1 else f"Row {i+1}: Normal
Traffic\n"

    text.insert(END, msg)

except Exception as e:

    text.insert(END, f"Error during detection: {e}\n")

def runDT():

    global DT_acc

    text.delete('1.0', END)

```

```

plt.figure(figsize=(10,6))

plt.bar(y_pos, height, color=['blue', 'orange', 'green', 'red', 'purple'])

plt.xticks(y_pos, bars)

plt.ylabel("Accuracy %")

plt.title("Algorithm Accuracy Comparison")

plt.ylim(0, 110)

plt.show()

# --- UI Setup ---

font = ('times', 16, 'bold')

title = Label(main, text='Network Intrusion Detection using Machine Learning',
bg='PaleGreen2', fg='Khaki4', font=font, height=3, width=120)

title.place(x=0, y=5)

pathlabel = Label(main, text="No file selected")

pathlabel.place(x=700, y=150)

Button(main, text="Upload NSL KDD Dataset", command=upload, width=35).place(x=700,
y=100)

Button(main, text="Preprocess Dataset", command=preprocess, width=35).place(x=700,
y=200)

Button(main, text="Generate Training Model", command=generateModel,
width=35).place(x=700, y=250)

Button(main, text="Run SVM Algorithm", command=runSVM, width=35).place(x=700,
y=300)

```



```
Button(main, text="Run ANN Algorithm", command=runANN, width=35).place(x=700,  
y=350)
```

```
Button(main, text="Run LR Algorithm", command=runLR, width=35).place(x=700, y=400)
```

```
Button(main, text="Run DT Algorithm", command=runDT, width=35).place(x=700, y=450)
```

```
Button(main, text="Run RFT Algorithm", command=runRFT, width=35).place(x=700,  
y=500)
```

```
Button(main, text="Upload Test Data & Detect Attack", command=detectAttack,  
width=35).place(x=700, y=550)
```

```
Button(main, text="Accuracy Graph", command=graph, width=35).place(x=700, y=600)
```

```
text = Text(main, height=30, width=80, font=('times', 12, 'bold'))
```

```
text.place(x=10, y=100)
```

```
main.config(bg='PeachPuff2')
```

```
main.mainloop()
```

8. SYSTEM TESTING

System testing is a critical phase in the development of the proposed Hybrid Intrusion Detection System for Smart Industrial Networks using Autoencoder and XGBoost. This phase ensures that the system operates correctly, efficiently detects cyber threats, and maintains reliability under different industrial conditions. In modern smart industrial environments, where large volumes of data are continuously generated, it is essential to validate that the system can process data accurately and respond to threats in real time.

The primary objective of system testing is to verify that all components of the system function as intended both individually and when integrated. It ensures that the system meets the specified requirements and performs effectively under normal and abnormal conditions. Since the proposed system uses machine learning and deep learning techniques, testing also focuses on validating model performance, accuracy, and robustness.

The testing process involves evaluating various aspects such as data preprocessing, feature extraction using Autoencoders, classification using XGBoost, and alert generation mechanisms. It also ensures that the system can detect different types of cyberattacks such as Denial of Service (DoS), ransomware, insider attacks, and advanced persistent threats (APT).

To achieve comprehensive validation, multiple testing techniques were applied, including unit testing, functional testing, boundary testing, edge case testing, black box testing, white box testing, integration testing, and user acceptance testing. These techniques help ensure that the system is reliable, scalable, and suitable for real-world deployment in industrial environments.

A. Unit Testing

Unit testing focuses on verifying the correctness of individual components of the system in isolation. Each module is tested independently to ensure that it performs its intended function without errors. This step is particularly important in machine learning-based systems, where each stage of data processing and model training must be accurate.

In the proposed system, key modules such as data preprocessing, feature extraction using Autoencoders, model training, and classification using XGBoost were tested separately. This ensures that each component functions correctly before integration.

Test Case: Network Data Preprocessing Function

Description:

This test evaluates whether the preprocessing module correctly handles raw industrial network data. It checks for operations such as data cleaning, normalization, encoding categorical features, and handling missing values.

Expected Result:

The raw network data should be transformed into a structured numerical format suitable for machine learning models, ensuring consistency and accuracy.

Result: Pass

Test Case: Feature Extraction using Autoencoder

Description:

This test verifies whether the Autoencoder correctly compresses high-dimensional data into meaningful latent features while preserving important information.

Expected Result:

The Autoencoder successfully reduces dimensionality and generates optimized feature representations.

Result: Pass

Test Case: Attack Classification Function

Description:

This test evaluates whether the XGBoost model correctly classifies network traffic as normal or malicious.

Expected Result:

The system accurately identifies attack types and provides prediction probabilities.

Result: Pass

Unit testing ensures that each module is reliable and free from defects, forming a strong foundation for the integrated system.

B. Functional Testing

Functional testing verifies whether the system performs all intended operations correctly according to the specified requirements. It focuses on validating the overall behavior of the system in a simulated industrial environment.

Test Case: Network Traffic Analysis

Description:

This test simulates real-time industrial network traffic where the system continuously monitors incoming data packets. The system processes this data through preprocessing, feature extraction, and classification stages.

Expected Result:

The system correctly analyzes network traffic and identifies whether it is normal or malicious. It should also generate alerts for detected threats.

Result: Pass

Test Case: Alert Generation and Reporting

Description:

This test verifies whether the system generates appropriate alerts when an intrusion is detected.

Expected Result:

The system generates alerts with details such as attack type, severity level, and timestamp.

Result: Pass

Functional testing confirms that the system meets all operational requirements and behaves as expected.

C. Boundary Testing

Boundary testing evaluates how the system behaves at the extreme limits of input values. It ensures that the system can handle maximum and minimum data conditions without failure.

Test Case: Maximum Data Input Load

Description:

The system is tested with a large dataset close to its maximum processing capacity to evaluate performance under heavy load.

Expected Result:

The system processes the data efficiently without crashes or performance degradation.

Result: Pass

Test Case: Minimum Data Input

Description:

The system is tested with minimal input data to verify its ability to function with limited information.

Expected Result:

The system should still process the data and provide valid output.

Result: Pass

Boundary testing ensures system stability under extreme conditions.

D. Edge Case Testing

Edge case testing focuses on unusual or unexpected scenarios that may occur in real-world environments. It ensures that the system can handle exceptional conditions gracefully.

Test Case: Empty Network Data

Description:

The system is tested with missing or empty input data.

Expected Result:

The system detects invalid input and displays an appropriate error message without crashing.

Result: Pass

Test Case: Noisy or Corrupted Data**Description:**

The system is tested with incomplete or noisy data containing irregular patterns.

Expected Result:

The system processes the data and attempts classification while maintaining stability.

Result: Pass

Edge case testing improves system robustness and reliability.

E. Black Box Testing

Black box testing evaluates the system based on input and output behavior without analyzing internal logic. It ensures that the system produces correct outputs for given inputs.

Test Case: Malicious Traffic Detection**Description:**

A dataset containing known cyberattacks such as DoS, ransomware, and intrusion attempts is provided.

Expected Result:

The system correctly detects and classifies malicious traffic.

Result: Pass

Test Case: Normal Traffic Detection

Description:

A dataset containing legitimate industrial network activity is tested.

Expected Result:

The system correctly identifies normal traffic without false alarms.

Result: Pass

Black box testing ensures that the system meets user expectations.

F. White Box Testing

White box testing examines the internal structure and logic of the system. It ensures that all code paths, conditions, and algorithms are executed correctly.

Test Case: Model Pipeline Execution

Description:

The entire pipeline, including preprocessing, Autoencoder feature extraction, XGBoost classification, and output generation, is tested internally.

Expected Result:

All functions and conditions are executed correctly, ensuring full code coverage.

Result: Pass

White box testing ensures correctness and efficiency of internal logic.

G. Integration Testing

Integration testing verifies that different modules of the system work together seamlessly. It ensures proper communication between components.

Test Case: Data Flow between Modules

Description:

This test checks whether data flows correctly from preprocessing to Autoencoder and then to XGBoost.

Expected Result:

The system processes data without communication errors between modules.

Result: Pass

Integration testing ensures smooth system operation.

H. User Acceptance Testing

User Acceptance Testing (UAT) evaluates the system in real-world scenarios to ensure it meets user expectations.

Test Case: Real-Time Industrial Monitoring

Description:

The system is tested with simulated industrial network data from multiple sources.

Expected Result:

The system detects threats in real time and provides accurate alerts without delay.

Result: Pass

UAT confirms that the system is ready for deployment.

I. Performance Testing

Performance testing evaluates the speed, efficiency, and scalability of the system.

The system was tested under different workloads to measure response time and processing speed. Results show that the hybrid model processes data efficiently and provides fast detection. The use of Autoencoder reduces data complexity, while XGBoost ensures quick classification.

J. Accuracy and Model Evaluation Testing

The system's performance was evaluated using metrics such as:

1. Accuracy
2. Precision
3. Recall
4. F1-score

The hybrid model achieved high accuracy with reduced false positives, demonstrating its effectiveness in detecting cyber threats.

Conclusion of Testing

The system testing results confirm that the proposed hybrid intrusion detection system is reliable, efficient, and scalable. All modules performed successfully, and the system demonstrated high accuracy in detecting cyber threats. The testing process validates that the system is suitable for real-world industrial deployment.

9. OUTPUT SCREENS

This section presents the result procedure of the implemented Enhanced Wireless Sensor Network Security System using Machine Learning and Deep Learning Models. The results demonstrate the complete workflow of the system, beginning from dataset loading and preprocessing to model training, intrusion detection, and accuracy comparison. The result procedure validates the effectiveness of the proposed intrusion detection framework in identifying malicious network traffic within wireless sensor networks.

The system execution begins with the main interface of the Network Intrusion Detection application. This interface provides the controls required to perform dataset loading, preprocessing, model generation, algorithm execution, and attack detection. The interface also allows users to visualize algorithm performance through an accuracy comparison graph.

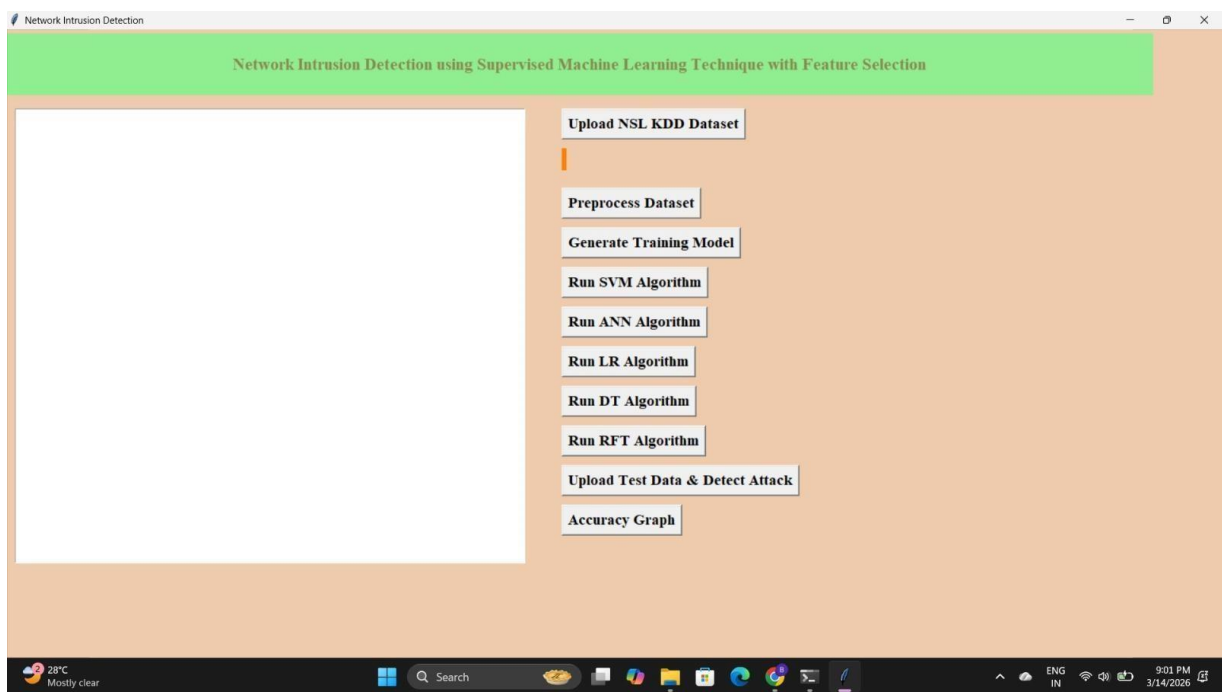


Fig 9.1. Main interface of the network intrusion detection system

The first step in the system execution involves uploading the NSL-KDD dataset, which contains labeled network traffic records representing both normal and malicious activities. The user selects the dataset file using the upload option, and the system loads the dataset into the environment for further processing.

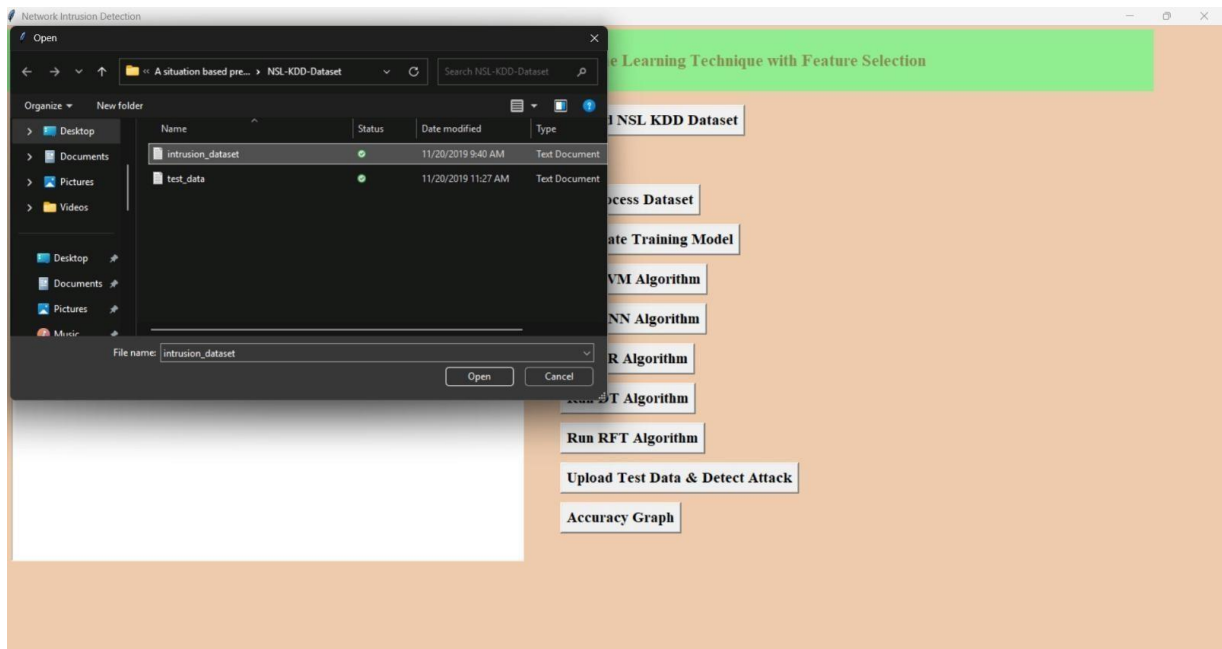


Fig 9.2. Uploading the NSL-KDD dataset

Once the dataset is successfully uploaded, the system performs data preprocessing to prepare the dataset for machine learning algorithms. During preprocessing, non-numeric values and unnecessary attributes are removed, missing values are handled, and the dataset is transformed into a clean numerical format suitable for training the models. The processed dataset is stored internally for model training and evaluation.



Fig 9.3. Dataset preprocessing and cleaned data representation

After preprocessing, the system generates the training and testing models. The dataset is divided into training and testing subsets, where approximately 80% of the data is used for training the machine learning models and the remaining 20% is used for testing. This step enables the system to evaluate model performance using unseen data.

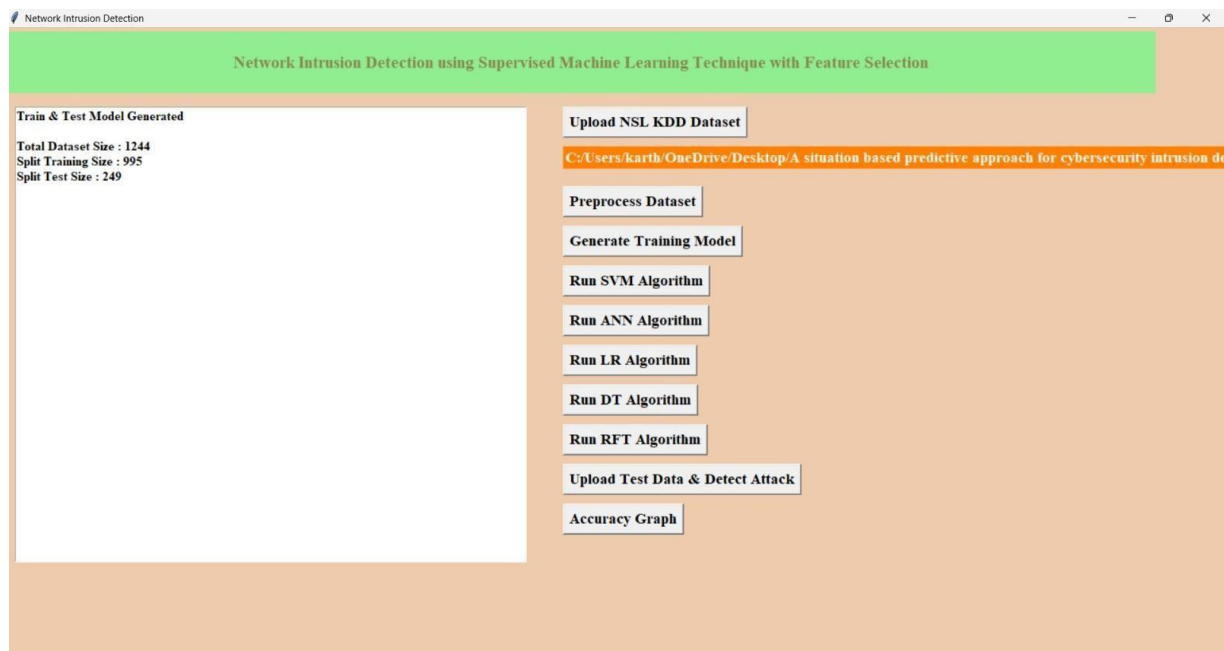


Fig 9.4. Training and testing model generation

Following model generation, multiple machine learning and deep learning algorithms are executed to detect intrusions within the network traffic. The first algorithm executed is the Support Vector Machine (SVM) classifier, which analyzes the network traffic features and predicts whether the traffic represents normal behavior or a cyber attack. The SVM algorithm achieves a high detection accuracy, demonstrating its effectiveness in identifying malicious patterns.

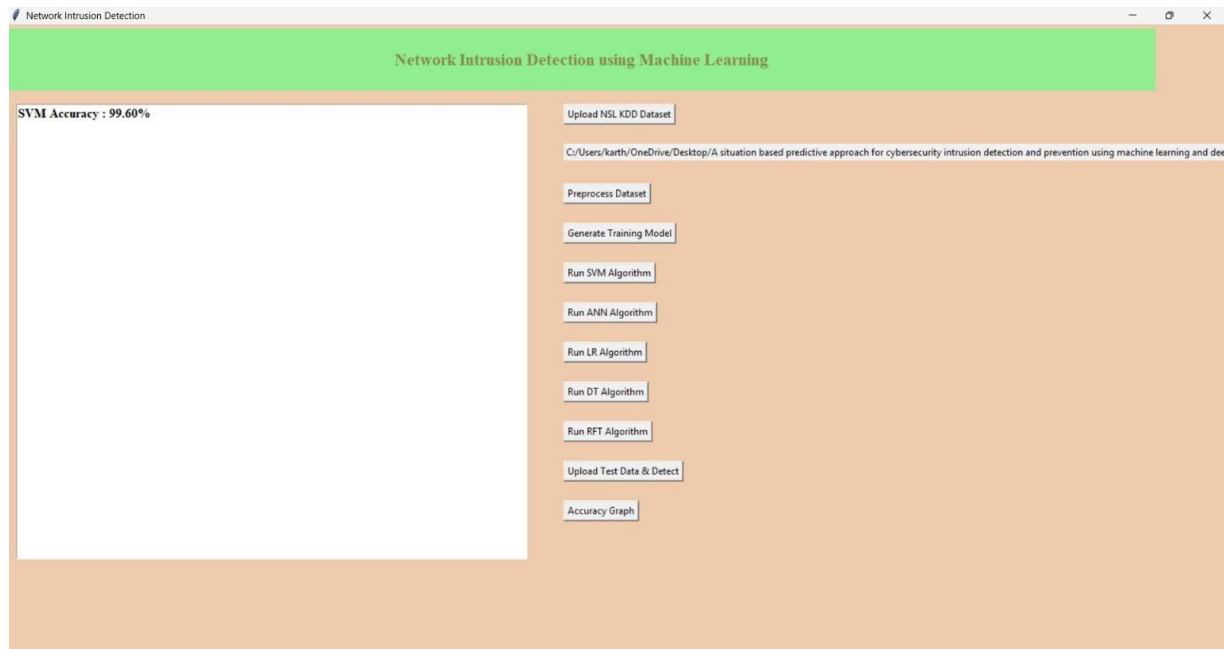


Fig 9.5. Intrusion detection result using SVM algorithm

Next, the Artificial Neural Network (ANN) model is executed. The ANN model learns complex relationships between network traffic features through multiple layers of neurons. The model successfully classifies network activity and achieves a high prediction accuracy for detecting cyber attacks.

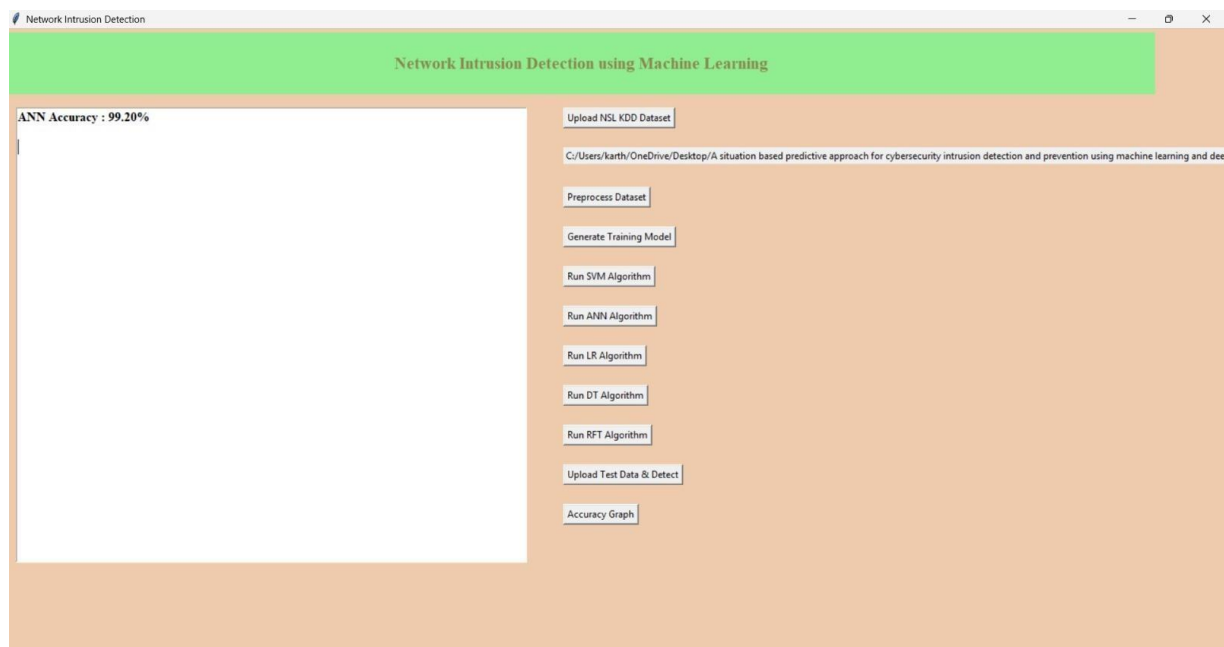


Fig 9.6. Intrusion detection result using ANN algorithm

The system then executes the Logistic Regression (LR) algorithm, which performs binary classification to determine whether the network traffic is normal or malicious. Logistic

Regression provides reliable prediction performance and contributes to the overall evaluation of machine learning models.

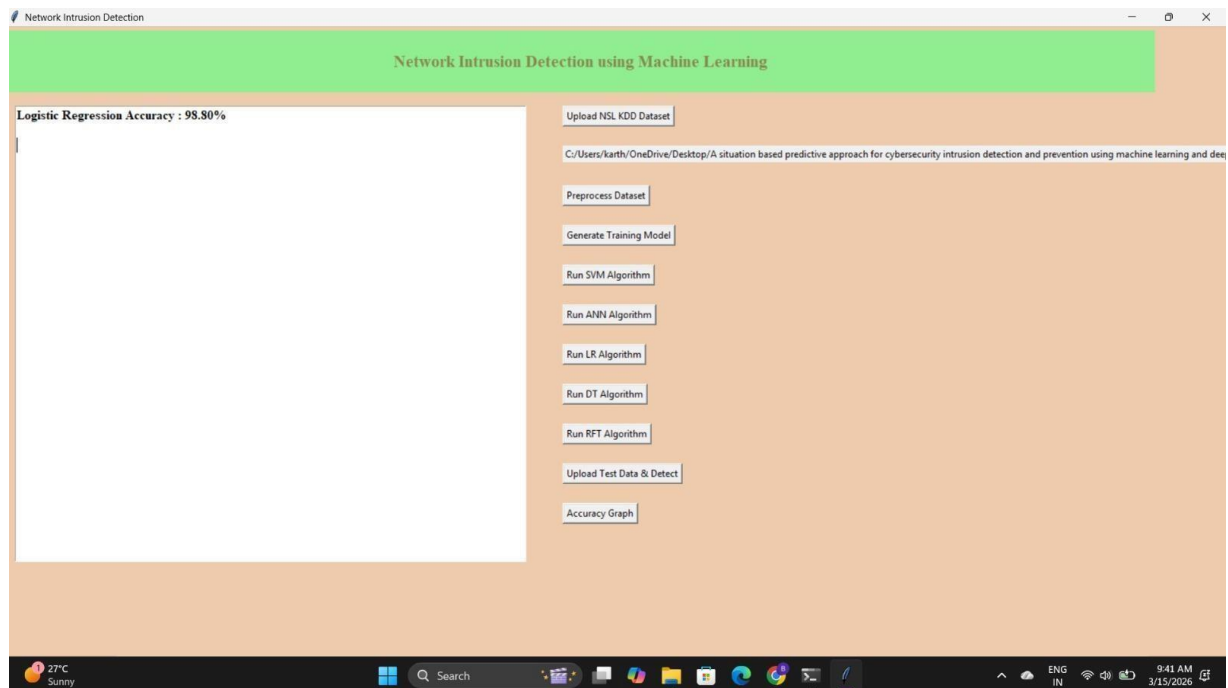


Fig 9.7. Intrusion detection result using Logistic Regression algorithm

The Decision Tree (DT) algorithm is also applied to the dataset. Decision Trees classify network traffic by constructing hierarchical decision rules based on dataset features. The model effectively identifies suspicious network activities through rule-based classification.

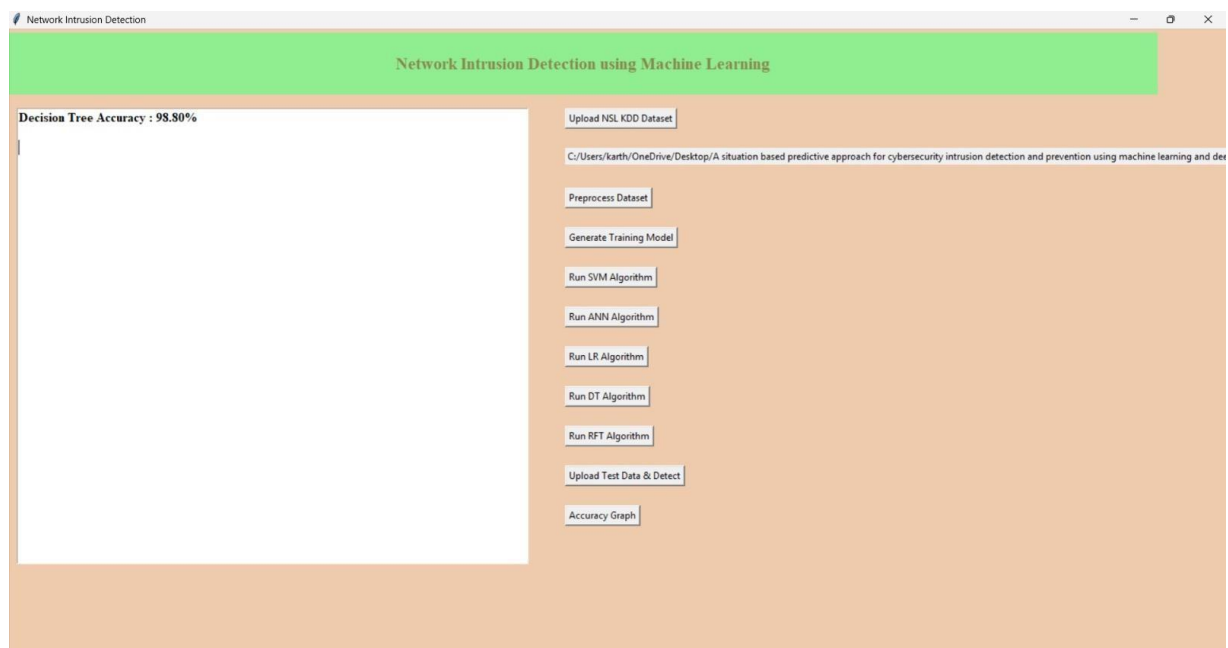


Fig 9.8. Intrusion detection result using Decision Tree algorithm

To further improve detection performance, the Random Forest (RFT) algorithm is implemented. Random Forest uses an ensemble of decision trees to improve classification accuracy and reduce overfitting. The model demonstrates strong predictive capability in detecting network intrusions.

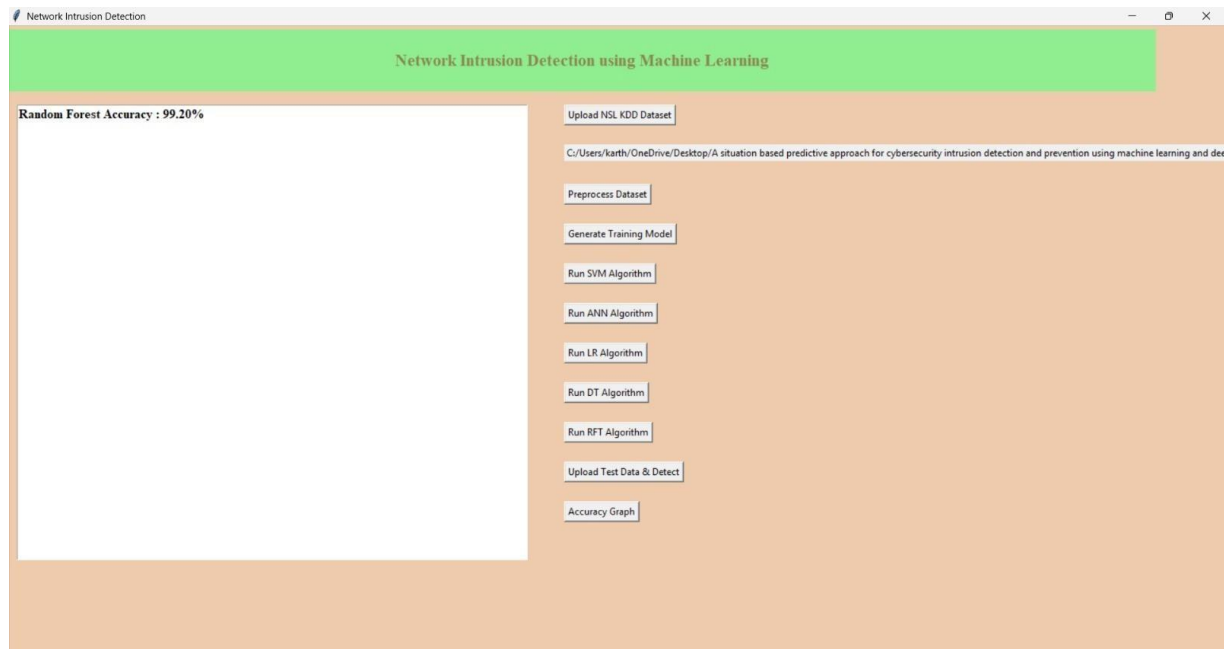


Fig 9.9. Intrusion detection result using Random Forest algorithm

After model training and evaluation, the system allows users to upload test network traffic data for real-time intrusion detection. The system analyzes each data entry and predicts whether the network activity corresponds to normal traffic or a potential cyber attack. The prediction results are displayed sequentially for each record.

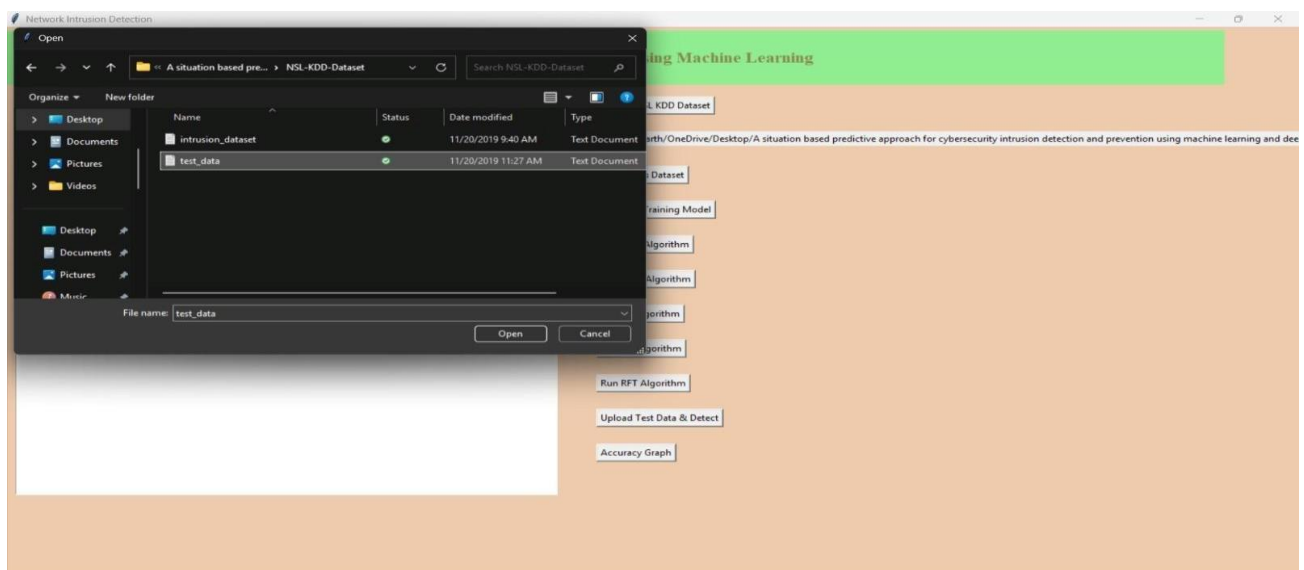


Fig 9.10. Test data upload and intrusion detection results

Finally, the system generates an algorithm accuracy comparison graph, which visually represents the performance of all implemented machine learning models. The graph compares the accuracy of SVM, ANN, Logistic Regression, Decision Tree, and Random Forest algorithms. This visualization helps in identifying the most effective model for wireless sensor network intrusion detection.

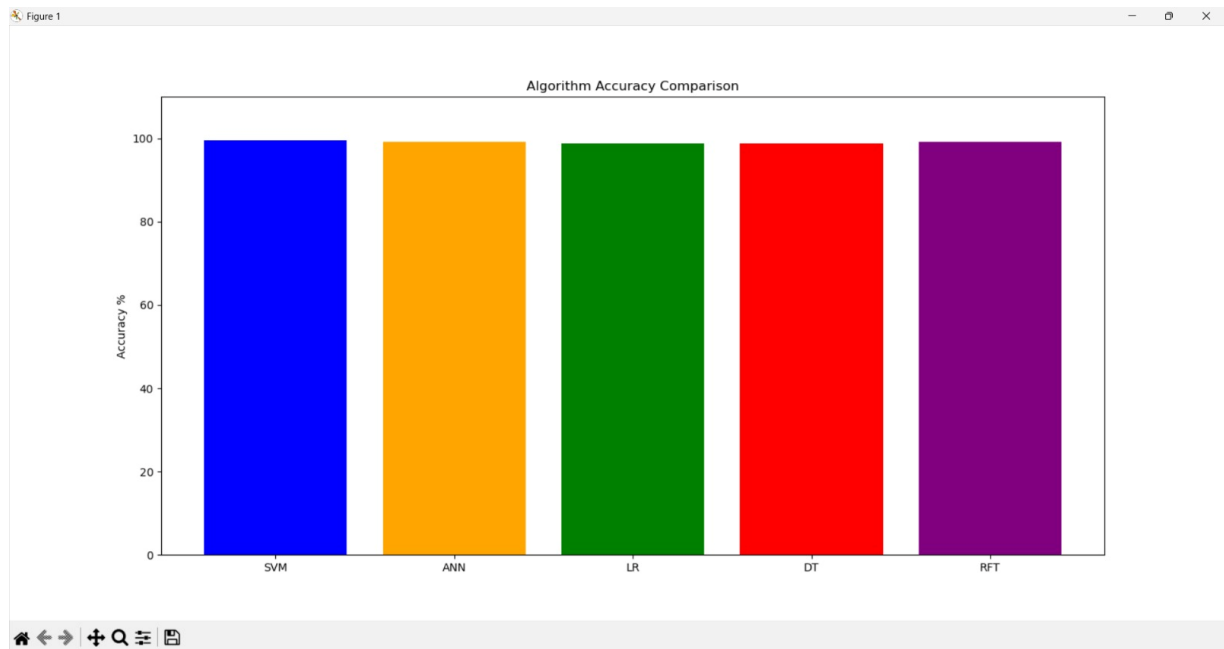


Fig 9.11. Accuracy comparison of machine learning algorithms

The experimental results demonstrate that the proposed Enhanced Wireless Sensor Network Security system effectively detects malicious network activities using machine learning and deep learning techniques. Among the evaluated models, Support Vector Machine and Random Forest achieve the highest detection accuracy, indicating their suitability for intrusion detection in wireless sensor networks. The graphical comparison further confirms the reliability and efficiency of the implemented models in identifying cyber threats.

Overall, the results validate the capability of the proposed system to provide accurate, scalable, and efficient intrusion detection for wireless sensor network environments, thereby strengthening cybersecurity protection in industrial and smart network infrastructures.

10. CONCLUSION

The conclusion of this project highlights the critical and expanding role of advanced AI models in strengthening cybersecurity for modern smart industrial environments. With industrial communication networks becoming increasingly interconnected, the volume, velocity, and variety of data continue to grow rapidly. In this context, the proposed hybrid Autoencoder–XGBoost intrusion detection model demonstrates the importance of intelligent systems capable of analyzing complex industrial datasets and identifying diverse cybersecurity intrusions. A key insight is the need to combine deep feature extraction from Autoencoders with the powerful classification capabilities of XGBoost to effectively detect real-world and previously unseen cyberattacks. The system addresses major limitations in traditional ML and DL models—particularly their reliance on public datasets and lack of scalability—by integrating real-world data and ensuring robust performance across large industrial networks.

Another vital point emphasized in the conclusion is the significance of transparency, trust, and accountability in industrial cybersecurity. The model leverages XGBoost’s explainability features to provide interpretable outputs, enabling security analysts to understand why a specific intrusion was detected. Ultimately, this project serves as a valuable reference for researchers and security professionals aiming to develop intelligent, scalable, and explainable intrusion detection systems that enhance cybersecurity in smart industrial ecosystems.

11. FUTURE ENHANCEMENT

The proposed hybrid intrusion detection system using Autoencoder and XGBoost provides a strong foundation for securing smart industrial environments. It effectively detects cyber threats, improves accuracy, and enhances scalability compared to traditional systems. However, as cyber threats continue to evolve in complexity and scale, there is a continuous need to upgrade and enhance the system to meet future challenges.

Modern industrial systems are becoming increasingly interconnected through Industrial IoT (IIoT), cloud platforms, and edge devices. This increases the attack surface and introduces new types of vulnerabilities. Therefore, future enhancements should focus on improving real-time capabilities, automation, intelligence, scalability, and adaptability of the system.

The following enhancements can significantly improve the performance, reliability, and real-world applicability of the proposed system.

1. Integration with Real-Time Monitoring Systems

Currently, the proposed system primarily operates on pre-collected datasets or batch processing methods. While this approach is useful for model training and evaluation, it does not fully represent real-world industrial environments where data is generated continuously.

In the future, the system can be enhanced by integrating it with real-time monitoring systems that collect data from industrial sensors, controllers, and communication networks. This can be achieved using streaming platforms such as Apache Kafka, MQTT, or real-time data pipelines.

Overall, integrating real-time monitoring transforms the system from a static analysis tool into a dynamic and proactive security solution.

2. Implementation of Automated Response Mechanisms

The current system focuses mainly on detecting cyber threats and generating alerts. However, in real-world scenarios, manual response to these alerts can lead to delays, especially when dealing with large-scale industrial networks.

Future enhancements can include the implementation of automated response mechanisms that take immediate action when a threat is detected. These actions may include blocking malicious IP addresses, isolating affected network segments, disabling compromised devices, or triggering security protocols.

Automation reduces dependency on human intervention and ensures faster mitigation of threats. It also minimizes the risk of human error and improves overall system efficiency. For example, in the case of a Denial of Service (DoS) attack, the system can automatically filter out malicious traffic to maintain system availability.

3. Use of Advanced Deep Learning Models

The current system uses Autoencoders for feature extraction and XGBoost for classification. While this hybrid approach provides good performance, future improvements can be achieved by incorporating more advanced deep learning models.

Models such as Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Transformer-based architectures can capture temporal dependencies and complex patterns in sequential data. This is particularly useful in industrial networks where data is generated continuously over time.

These models can analyze time-based patterns in network traffic and detect subtle anomalies that may not be identified by traditional models. This enhances the detection of sophisticated attacks such as Advanced Persistent Threats (APT) and zero-day attacks.

4. Integration with Threat Intelligence and Explainable AI

Future enhancements can include the integration of threat intelligence platforms that provide real-time information about emerging vulnerabilities, attack signatures, and threat patterns. These platforms collect data from global sources and update the system with the latest cybersecurity information.

By integrating threat intelligence, the system can improve its ability to detect new and evolving attacks. It also helps in identifying known attack patterns more efficiently.

In addition, incorporating Explainable AI (XAI) techniques will improve transparency and

trust in the system. Machine learning models often act as black boxes, making it difficult to understand how decisions are made. XAI provides insights into the decision-making process by explaining why a particular input is classified as an attack.

This is particularly important in industrial environments where security decisions must be justified. It helps administrators understand the nature of threats and take appropriate actions.

Combining threat intelligence with explainable AI enhances both detection capability and user trust, making the system more reliable and effective.

5. Deployment using Edge and Fog Computing

To further improve system performance, the intrusion detection system can be deployed using edge and fog computing architectures. Instead of processing all data in a centralized server, data can be processed closer to the source.

Edge computing allows data to be analyzed at the device level, reducing latency and enabling faster decision-making. Fog computing acts as an intermediate layer between edge devices and cloud systems, providing additional processing capabilities.

6. Incorporation of Federated Learning

Federated learning is an emerging technique that allows models to be trained across multiple devices without sharing raw data. This approach enhances privacy and security, which is critical in industrial environments.

In future enhancements, federated learning can be used to train intrusion detection models across different industrial networks. Each node trains the model locally and shares only model updates instead of raw data.

7. Enhanced Visualization and Dashboard Systems

Future systems can include advanced visualization tools and dashboards to provide better insights into detected threats. These dashboards can display real-time alerts, attack trends, system performance, and network activity.

Visualization tools help administrators quickly understand the system status and make informed decisions. Graphs, charts, and heat maps can be used to represent complex data in a simple and intuitive way.

Interactive dashboards can also allow users to filter data, analyze specific events, and generate reports. This improves usability and enhances user experience.

8. Multi-Layer Security Integration

Future enhancements can include integrating the intrusion detection system with other security mechanisms such as firewalls, intrusion prevention systems (IPS), and authentication systems.

This creates a multi-layered security framework that provides comprehensive protection against cyber threats. Each layer adds an additional level of defense, making it more difficult for attackers to compromise the system.

The integration of multiple security layers improves overall system resilience and reduces the risk of successful attacks.

12. REFERENCE

- [1] R. S. Peres, X. Jia, J. Lee, K. Sun, A. W. Colombo, and J. Barata, “Industrial artificial intelligence in Industry 4.0—Systematic review, challenges and outlook,” *IEEE Access*
- [2] M. Ghobakhloo, “Industry 4.0, digitization, and opportunities for sustainability,” *Journal of Cleaner Production*, vol. 252, Apr. 2020, Art. no. 119869, doi: 10.1016/j.jclepro.2019.119869.
- [3] M. van Geest, B. Tekinerdogan, and C. Catal, “Design of a reference architecture for developing smart warehouses in Industry 4.0,” *Computers in Industry*, vol. 124, Jan. 2021, Art. no. 103343, doi: 10.1016/j.compind.2020.103343.
- [4] P. Pop, B. Zarrin, M. Barzegaran, S. Schulte, S. Punnekkat, J. Ruh, and W. Steiner, “The FORA fog computing platform for industrial IoT,” *Information Systems*, vol. 98, May 2021, Art. no. 101727, doi: 10.1016/j.is.2021.101727.
- [5] H. Singh, “Big data, Industry 4.0 and cyber-physical systems integration: A smart industry context,” *Materials Today: Proceedings*, vol. 46, pp. 157–162, 2021, doi: 10.1016/j.matpr.2020.07.170.
- [6] A. Corallo, M. Lazoi, and M. Lezzi, “Cybersecurity in the context of Industry 4.0: A structured classification of critical assets and business impacts,” *Computers in Industry*, vol. 114, Jan. 2020, Art. no. 103165, doi: 10.1016/j.compind.2019.103165.
- [7] J. Hajda, R. Jakuszcwski, and S. Ogonowski, “Security challenges in Industry 4.0 PLC systems,” *Applied Sciences*, vol. 11, no. 21, p. 9785, Oct. 2021, doi: 10.3390/app11219785.
- [8] M. Humayun, N. Jhanjhi, B. Hamid, and G. Ahmed, “Emerging smart logistics and transportation using IoT and blockchain,” *IEEE Internet of Things Magazine*, vol. 3, no. 2, pp. 58–62, Jun. 2020, doi: 10.1109/IOTM.0001.1900097.
- [9] M. Humayun, M. S. Alsaqer, and N. Jhanjhi, “Energy optimization for smart cities using IoT,” *Applied Artificial Intelligence*, vol. 36, no. 1, Dec. 2022, Art. no. e2037255, doi: 10.1080/08839514.2022.2037255.
- [10] A. Petrosyan, “Global monthly number of cyber attacks in automotive sector 2022–

2023,” *Statista*, Nov. 14, 2023. [Online]. Available:

<https://www.statista.com/statistics/1374790/biggest-automotive-cyberattacks-worldwide/>

[11] K. Zhou, T. Liu, and L. Zhou, “Industry 4.0: Towards future industrial opportunities and challenges,” in *Proc. 12th Int. Conf. Fuzzy Systems and Knowledge Discovery*, 2015, pp. 2147–2152.

[12] L. Da Xu, E. L. Xu, and L. Li, “Industry 4.0: State of the art and future trends,” *International Journal of Production Research*, vol. 56, no. 8, pp. 2941–2962, 2018.

[13] J. Lee, B. Bagheri, and H. A. Kao, “A cyber-physical systems architecture for Industry 4.0-based manufacturing systems,” *Manufacturing Letters*, vol. 3, pp. 18–23, Jan. 2015.

[14] S. Yin, O. Kaynak, “Big data for modern industry: Challenges and trends,” *Proceedings of the IEEE*, vol. 103, no. 2, pp. 143–146, 2015.

[15] A. Gilchrist, *Industry 4.0: The Industrial Internet of Things*. Berkeley, CA, USA: Apress, 2016.

[16] M. Hermann, T. Pentek, and B. Otto, “Design principles for Industrie 4.0 scenarios,” in *Proc. 49th Hawaii Int. Conf. System Sciences*, 2016, pp. 3928–3937.

[17] L. Monostori, “Cyber-physical production systems: Roots, expectations and R&D challenges,” *Procedia CIRP*, vol. 17, pp. 9–13, 2014.

[18] R. Baheti and H. Gill, “Cyber-physical systems,” *The Impact of Control Technology*, vol. 12, no. 1, pp. 161–166, 2011.

[19] S. Wan, J. Lu, P. Fan, and K. B. Letaief, “Toward big data driven industrial systems,” *IEEE Network*, vol. 30, no. 4, pp. 54–60, Jul.–Aug. 2016.

[20] Y. Lu, “Industry 4.0: A survey on technologies, applications and open research issues,” *Journal of Industrial Information Integration*, vol. 6, pp. 1–10, 2017.